

GroupBuy: A Query Framework for Constrained Multi-Item Retrieval: Technical Report

Yueting Chen
Seattle University
Seattle, USA
ychen24@seattleu.edu

Kevin Chen
University of Toronto
Toronto, Canada
kckevin.chen@mail.utoronto.ca

Xiaohui Yu
York University
Toronto, Canada
xhyu@yorku.ca

Nick Koudas
University of Toronto
Toronto, Canada
koudas@cs.toronto.edu

ABSTRACT

Conversational shopping agents powered by large language models (LLMs) allow users to express requests for several items in natural language. An LLM can translate such a request into structured item requirements, but a query processing challenge remains. The retrieval system must select one item per requirement so that the complete bundle satisfies shared constraints such as a total budget. Independent ranked retrieval evaluates items separately and does not enforce these constraints across the bundle. We formalize this task as *Constraint Group Query* (CGQ), which selects exactly one item per requirement from semantically retrieved candidate sets, subject to a matching quality threshold and a shared relaxed budget cap. When preference weights are unknown, Pareto CGQ returns one representative assignment per nondominated objective vector, capturing tradeoffs among solution quality, cost overhead, and total rating. We give CGQ instances with exponentially large Pareto frontiers. We combine Pareto dynamic programming, candidate reduction, and sound partial assignment dominance based on raw accumulated cost to compute exact frontiers. Pareto Beam Search (PBS) retains a bounded number of partial assignments at each search level. Pareto Tree Search (PTS) maintains the current frontier while exploring partial assignments in a tree. We establish the conditions under which each method returns the complete Pareto frontier. Both methods extend to other objectives supporting the same dominance and bounding operations. Experiments on Amazon and Airbnb show that PBS without beam truncation recovers the complete frontier, while bounded search reduces tail latency on Airbnb. With query embeddings precomputed, median latency for candidate retrieval and CGQ processing is below 30 ms on Amazon catalogs containing up to 2.14 million items.

PVLDB Reference Format:

Yueting Chen, Kevin Chen, Xiaohui Yu, and Nick Koudas. GroupBuy: A Query Framework for Constrained Multi-Item Retrieval: Technical Report. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/yestincheng/groupbuy>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

The rise of shopping agents powered by large language models (LLMs) is changing how users interact with retrieval systems [22, 37]. Instead of submitting a keyword query and examining a ranked list, users can express intent involving several items. For example, a user may ask for a charger, noise cancelling headphones, and a laptop backpack for a work trip, all within a total budget. The user expects the system to return a complete bundle that is ready to purchase [13, 15]. The same query pattern arises in travel planning. A user visiting several cities may request one suitable accommodation in each city under a total lodging budget.

Traditional ranked retrieval returns a list of items ranked independently by their relevance to the query [28]. A request involving several items changes the retrieval task. The challenge is no longer to find the best charger in isolation. It is to find a set of items that jointly satisfies the individual requirements and global constraints. Once constraints apply to the bundle, items that are best individually may form an infeasible or poor bundle when combined. From a query processing perspective, the LLM therefore produces a structured request containing several item requirements and global constraints. The retrieval system must select a complete bundle rather than return a ranked list. This setting naturally extends both top- k retrieval and package queries [5, 6, 24].

We formalize this setting as the *Constraint Group Query* (CGQ) problem. A CGQ consists of multiple item requirements and constraints at two levels. An item level constraint requires the item selected for each requirement to meet a minimum matching quality. A global constraint, such as a total budget, applies to the complete bundle. A CGQ selects exactly one item for each requirement while satisfying both types of constraints. Matching quality allows CGQ to accommodate semantic requirements without exact matches.

Per requirement budgets are appropriate when the user supplies separate spending limits, and they make budget feasibility separable across requirements. We consider requests that specify only a total budget, leaving its allocation across requirements to item selection. Imposing separate budgets on these requests adds constraints that the user did not specify. A fixed split can exclude bundles that satisfy the total budget because spending less on one requirement does not increase the limits for the others. In shopping, a less expensive charger may leave more budget for headphones with higher matching quality or rating. In travel, a less expensive stay in one city may offset a more expensive stay in another.

CGQ is related to package queries [6] because both select groups of items under collective constraints. CGQ organizes its search space by requirement within this setting. Each requirement defines its own candidate set, exactly one item must be selected from

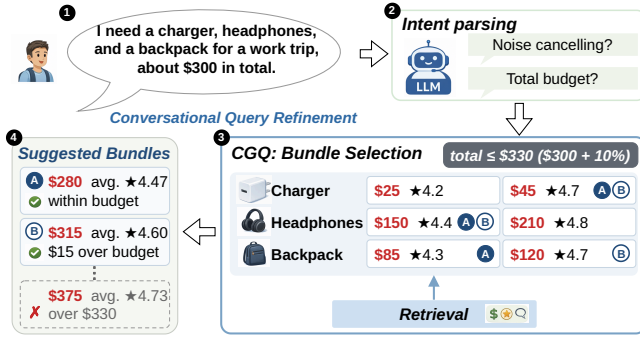


Figure 1: CGQ in a shopping pipeline. Bundles A and B trade cost for rating, and the dashed one exceeds \$330.

each set, and an item’s matching quality depends on the requirement. CGQ therefore couples semantic retrieval with constrained assignment across multiple requirements.

In practice, a bundle can be evaluated from several aspects, such as how well its items match the requirements and how users rate them. The relative importance of these aspects is unknown in most cases because users rarely provide explicit preference weights. We therefore define Pareto CGQ, which returns bundles that satisfy the item requirements and global constraints while capturing different tradeoffs among the objectives, where no returned bundle is uniformly better than another [4, 17]. A shopping agent can present these alternatives before the user selects a bundle. When preference weights are known, the weighted variant, Weighted CGQ, returns a single optimal bundle. Pareto CGQ thus preserves the tradeoffs before the user specifies how matching quality, cost, and rating should be weighted. The agent can present selected alternatives from the frontier or rank them after obtaining additional preferences.

Figure 1 shows how CGQ fits into a complete shopping pipeline. An LLM converts a user’s request into individual item requirements and obtains any missing constraints. A retrieval system then gathers candidate items for each requirement [14, 15, 22, 37]. CGQ operates on these candidate sets and selects a bundle that satisfies the global constraints. Our work focuses on this final selection stage. The pipeline also applies to other domains, including travel planning, with one accommodation requirement per destination.

For Weighted CGQ, we formulate an integer linear program (ILP) that returns the optimal bundle when the preference weights are known. Pareto CGQ presents a different challenge. Large Pareto frontiers are a known limitation of multiobjective optimization [2]. We give a CGQ instance whose complete frontier can contain exponentially many objective vectors, making the output size, in addition to the Cartesian product of the candidate sets, a fundamental source of complexity. Exhaustive enumeration can quickly become intractable as the number of requirements grows. One approach repeatedly solves the problem to obtain different frontier points, but it may miss some tradeoffs or require many separate runs [26]. We therefore develop methods that find multiple frontier points in a single run.

Our exact search combines Pareto dynamic programming [3, 19] with candidate reduction and dominance based on raw cost. To limit

search work, we use Pareto Beam Search (PBS) [23], which constructs partial assignments one requirement at a time and prunes them as it proceeds. PBS retains the raw accumulated cost when comparing partial assignments so that partial assignments with different remaining budgets are distinguished. Without beam truncation, PBS returns the complete frontier, while a bounded beam limits search work at the cost of some frontier recovery. We also propose Pareto Tree Search (PTS), which uses depth first search with dominance and bound pruning, adapting multicriteria branch and bound [12] to requirement assignments. PTS organizes partial assignments in a tree, returns to earlier choices, and prunes subtrees that cannot improve its current frontier. It maintains the current frontier and records completed subtrees, allowing it to return the alternatives found whenever the search stops while avoiding repeated work. The comparison of partial assignments follows a sound dominance rule based on raw accumulated cost. The rule accounts for the budget remaining for later requirements and preserves Pareto optimal completions. PBS and PTS use this result while retaining search state in complementary ways. PBS bounds the partial assignments retained at each search level, whereas PTS preserves explored choices in its tree for later expansion.

Our contributions are summarized as follows.

- We formulate CGQ under shared constraints, with Pareto and weighted variants. We give a CGQ instance with an exponentially large frontier for two objectives and show that Weighted CGQ is weakly NP-hard for integer prices.
- We combine Pareto DP with candidate reduction and sound partial assignment dominance based on raw cost. PBS and PTS bound search work for queries with high tail latency, and PTS maintains its frontier throughout search. We establish their completeness conditions.
- We evaluate exact and bounded search end to end on Amazon and Airbnb, measuring frontier recall, hypervolume, median and tail latency, and PTS frontier quality over time.

2 PROBLEM FORMULATION

2.1 Constraint Group Query

We model each catalog entry as an item o with an attribute tuple A_o . We use $c_o \in \mathbb{R}_{\geq 0}$ for its price and $r_o \in \mathbb{R}_{\geq 0}$ for its rating. The tuple may contain additional descriptive attributes such as title, vendor, category, and location. Let $\mathcal{O}$ denote the full item catalog.

An item requirement q_i describes one item requested by the user. It typically contains a short natural language description, such as “a compact USB-C wall charger that can charge a phone quickly,” and may include structured predicates such as brand or category filters. We quantify how well an item satisfies a requirement using the following matching function.

Definition 2.1 (Matching Quality). The matching quality of an item o for an item requirement q_i is $f_m(q_i, o) \in [0, 1]$, where a higher value indicates a stronger match.

Matching quality represents the degree to which an item satisfies a requirement instead of treating matching as a binary result. A perfectly matching item may be absent from the catalog, unavailable among the retrieved candidates, or too expensive to combine with the items selected for other requirements. A candidate with

lower matching quality may still permit a feasible assignment or leave more budget for other requirements. We treat f_m as an input function. Possible implementations include embedding similarity and learned relevance models.

A user request contains a sequence of item requirements $G = \langle q_1, \dots, q_n \rangle$ and a target budget B . Unlike independent retrieval, which can answer each requirement separately, the system must return an item assignment $O_G = \langle o_{q_1}, \dots, o_{q_n} \rangle$ that assigns one item to every requirement. The selected items form the bundle presented to the user. An assignment in which every item matches perfectly and the total cost does not exceed B may not exist because matching items can be unavailable or collectively exceed the target budget. We use two parameters to define the acceptable matching quality and maximum total cost. The matching-quality threshold $\beta \in [0, 1]$ specifies the minimum matching quality of every selected item. The budget-relaxation parameter $\alpha \geq 0$ defines the relaxed budget cap $(1 + \alpha)B$. Together, β and the relaxed budget cap bound the permitted deviations in matching quality and total cost. Retrieval supplies candidates for each requirement, and C_i denotes those satisfying the structured predicates and matching threshold.

Definition 2.2 (Constraint Group Query). A *Constraint Group Query* (CGQ) is a tuple (G, B, α, β) . A solution $O_G = \langle o_{q_1}, \dots, o_{q_n} \rangle$ is *feasible* if $o_{q_i} \in C_i$ for every $q_i \in G$, which ensures $f_m(q_i, o_{q_i}) \geq \beta$, and

$$\sum_{i=1}^n c_{o_{q_i}} \leq (1 + \alpha)B.$$

A CGQ may be infeasible for two reasons. If some candidate set C_i is empty, no item satisfies that requirement at the current matching threshold, so the agent may revise the requirement or lower β . Otherwise, the minimum possible assignment cost is $C_{\min} = \sum_i \min_{o \in C_i} c_o$. If $C_{\min} > (1 + \alpha)B$, the system can report $C_{\min}$ as the minimum required relaxed budget, allowing the agent to ask the user whether to revise the budget.

Feasibility alone does not determine which assignment is most useful. One assignment may match the requirements more closely, while another may contain items with higher ratings or remain closer to the target budget. We therefore compare feasible assignments using three objectives. The solution quality is the average matching quality

$$Q(O_G) = \frac{1}{n} \sum_{i=1}^n f_m(q_i, o_{q_i}).$$

It captures how well the assignment satisfies the requirements overall. The total cost is $C(O_G) = \sum_{i=1}^n c_{o_{q_i}}$. Because α allows a feasible assignment to exceed the target budget, we measure this excess using the cost overhead $\Delta C(O_G) = \max(C(O_G) - B, 0)$. This objective distinguishes assignments that remain within the target from those that use some of the permitted tolerance. Finally, the total rating $R(O_G) = \sum_{i=1}^n r_{o_{q_i}}$ captures the aggregate ratings of the selected items. Thus, assignments are evaluated by maximizing Q and R while minimizing ΔC .

2.2 Multiple Objectives

The formulation supports any collection of objectives for which dominance is defined. We use solution quality, cost overhead, and

total rating as a concrete instantiation. The three objectives capture complementary aspects of an assignment. An assignment with higher solution quality may cost more, while a less expensive assignment may contain items with lower matching quality or ratings. Consequently, feasible assignments may represent different tradeoffs, and no single assignment is necessarily best in every objective.

To capture these tradeoffs, we compare feasible assignments using the objective vector $(Q(O_G), -\Delta C(O_G), R(O_G))$. A feasible assignment dominates another if it is at least as good in every objective and strictly better in at least one. Improving one objective of a nondominated assignment requires worsening another.

Definition 2.3 (Pareto CGQ). *Pareto CGQ* seeks the set of all non-dominated solutions to

$$\begin{aligned} \max_{O_G} \quad & (Q(O_G), -\Delta C(O_G), R(O_G)) \\ \text{s.t.} \quad & f_m(q_i, o_{q_i}) \geq \beta, \quad \forall q_i \in G, \\ & \sum_{i=1}^n c_{o_{q_i}} \leq (1 + \alpha)B, \end{aligned}$$

where $O_G = \langle o_{q_1}, \dots, o_{q_n} \rangle$ selects one candidate for each requirement. The nondominated solutions form the Pareto frontier.

The Pareto frontier gives the shopping agent a set of nondominated alternatives instead of forcing it to select one aspect as the sole criterion. The agent may present these alternatives to the user or pass them to a later decision process.

The frontier separates query processing from preference selection. After receiving it, the agent may filter the assignments using preferences expressed during the conversation, present a small set of representative tradeoffs, or rank the returned assignments using known weights. We do not study how the agent elicits these preferences or chooses which alternatives to present.

Returning the complete Pareto frontier also creates a computational challenge. Even a compact CGQ instance can contain many mutually nondominated assignments, and a complete algorithm must return all of them. Large Pareto frontiers are a known limitation of multiobjective optimization [2]. The following theorem gives a CGQ instance with an exponentially large frontier.¹

THEOREM 2.4 (EXPONENTIAL PARETO FRONTIER). *There are CGQ instances with n requirements whose Pareto frontier under $(Q, -\Delta C, R)$ contains 2^n objective vectors. Consequently, no algorithm can enumerate the frontier in time polynomial in the input size, independently of whether $P = NP$.*

PROOF SKETCH. Each requirement has two zero-cost candidates, one contributing to quality and the other to rating. We choose their contributions using powers of two so that the 2^n assignments have distinct objective vectors and higher quality always implies lower rating. Thus, all assignments are mutually nondominated. $\square$

The exponential size does not depend on the rating objective, and it already bounds methods that return one assignment per optimization run.

COROLLARY 2.5 (TWO OBJECTIVES SUFFICE). *There are CGQ instances with n requirements whose Pareto frontier under $(Q, -\Delta C)$*

¹Full proofs and complexity derivations appear in the technical report [1].

alone contains 2^n objective vectors with pairwise distinct cost overheads. On these instances, any method that returns one assignment per ILP run, including the ϵ constraint method, needs at least 2^n runs to return the complete frontier.

PROOF SKETCH. Each requirement offers a zero-cost candidate with zero quality and a candidate whose quality and price are proportional to a distinct power of two. Choose the budget below the smallest positive price and the relaxed cap to make every assignment feasible. The 2^n assignments then have distinct quality values, with higher quality always requiring greater cost overhead, so all are nondominated. A method returning one assignment per run therefore needs at least 2^n runs. $\square$

2.3 Weighted CGQ

In some cases, the user provides preferences explicitly or the system obtains them through prior interaction. Weighted CGQ represents these preferences as weights on the three objectives and returns one optimal feasible assignment.

The objectives use different units and ranges. Solution quality lies in $[0, 1]$, cost overhead is measured in currency, and total rating grows with the number of requirements. Combining their raw values would cause their numerical scales to affect the result independently of the preference weights. We therefore use solution quality directly, normalize cost overhead by its feasible range αB , and normalize total rating by $R_{\max} = nr_{\max}$, where $r_{\max}$ is the maximum catalog rating. We define

$$F(O_G) = \lambda_1 Q(O_G) - \lambda_2 \frac{\Delta C(O_G)}{\alpha B} + \lambda_3 \frac{R(O_G)}{R_{\max}}, \quad (1)$$

where $\lambda_1, \lambda_2, \lambda_3 \geq 0$ are the known preference weights. When $\alpha = 0$, feasibility forces $\Delta C = 0$, and the overhead term is omitted.

Definition 2.6 (Weighted CGQ). *Weighted CGQ* returns a feasible assignment O_G that maximizes $F(O_G)$.

Weighted CGQ generalizes the Multiple-Choice Knapsack Problem (MCKP) [30]. Each requirement defines a candidate group, and a feasible assignment selects one candidate from each group under a shared budget. With a strict budget and an objective based only on ratings, prices correspond to weights and ratings correspond to profits. This correspondence establishes the complexity of Weighted CGQ.

THEOREM 2.7 (COMPLEXITY OF WEIGHTED CGQ). *Given a threshold τ , deciding whether there exists a feasible assignment O_G such that $F(O_G) \geq \tau$ is NP-complete. Consequently, finding an optimal solution to Weighted CGQ is NP-hard.*

PROOF SKETCH. Feasibility and objective value can be checked in polynomial time, so the decision problem belongs to NP. The mapping above gives a polynomial reduction from Multiple Choice Knapsack. Scaling its profit threshold by the rating normalization preserves the answer, establishing NP hardness. $\square$

As in Multiple Choice Knapsack, the numeric budget capacity determines the size of a dynamic programming table for Weighted CGQ. The weighted objective depends on an assignment only through additive terms and the total cost, so integer prices allow the table to be indexed by total cost. This yields a polynomial time

algorithm when the integer capacity is polynomially bounded in the input length.

PROPOSITION 2.8 (PSEUDOPOLYNOMIAL WEIGHTED CGQ). *Suppose all numeric inputs are rational values encoded in binary, and all prices are nonnegative integers. Let $K = \lfloor (1 + \alpha)B \rfloor$. Weighted CGQ can be solved exactly using $O((K + 1) \sum_{i=1}^n |C_i|)$ arithmetic operations and comparisons, while storing $O(n(K + 1))$ table entries. The algorithm is pseudopolynomial in K , so the integer price formulation of Weighted CGQ is NP-hard only in the weak sense.*

PROOF SKETCH. Let $A(O_G) = \lambda_1 Q(O_G) + \lambda_3 R(O_G)/R_{\max}$ be the additive part of F , so that F depends on an assignment only through A and the total cost C . For $i \in \{0, \dots, n\}$ and an integer cost $c \leq K$, let $T[i][c]$ be the maximum of A over partial assignments of $q_1, \dots, q_i$ with total cost exactly c . Row i follows from row $i - 1$ by trying every candidate in C_i , and the optimum of Weighted CGQ is $\max_c (T[n][c] - \lambda_2 \max(c - B, 0)/(\alpha B))$, with the overhead term omitted when $\alpha = 0$. $\square$

The same dynamic program handles rational prices by scaling prices and the budget by a common denominator, denoted by D . This preserves feasibility and the objective, and replaces K with $K_D = \lfloor D(1 + \alpha)B \rfloor$. The scaled capacity can be large even when the original prices are small. Accounting for arithmetic costs, the running time is polynomial in the input bit length and K_D . It is therefore polynomial in the input length when K_D is polynomially bounded in that length.

3 METHODS

3.1 From Weighted ILP to Pareto Search

We begin with Weighted CGQ, whose single objective leads directly to an exact ILP. We then consider repeated ILP runs for obtaining multiple frontier points for Pareto CGQ and their limitations.

3.1.1 Solving Weighted CGQ with ILP. Weighted CGQ selects one candidate for each requirement while enforcing a shared budget, so its choices and constraints map naturally to an integer linear program (ILP). Let C_i contain the candidates for requirement q_i that satisfy the matching threshold. For each $o \in C_i$, we introduce a binary variable $x_{i,o}$ that indicates whether o is selected. An equality constraint selects exactly one candidate for each requirement, and a budget constraint limits the total cost to $(1 + \alpha)B$.

The solution quality and total rating objectives are weighted sums of the selected candidates and thus linear in $x_{i,o}$. The only nonlinear term is the cost overhead $\Delta C(O_G) = \max(C(O_G) - B, 0)$. We introduce a nonnegative variable y that is at least the selected cost minus B . When the cost overhead has a positive weight, the objective makes y equal to the overhead at optimality. Substituting the above into $F(O_G)$ gives an exact ILP for Weighted CGQ.²

3.1.2 Repeated ILP for Pareto CGQ. The weighted ILP returns one assignment for chosen weights. A natural approach to Pareto CGQ is to solve the ILP repeatedly with different weights. However, this approach can recover only supported points, which maximize some linear combination of the objectives. It may miss unsupported frontier points that do not maximize any such combination.

²The complete formulation appears in the technical report [1].

The ϵ constraint method addresses this limitation by optimizing one objective while placing bounds on the others [26]. For Pareto CGQ, each threshold pair (ϵ_C, ϵ_R) defines an ILP that maximizes $Q(O_G)$ subject to the CGQ feasibility constraints, the cost overhead cap $\Delta C(O_G) \leq \epsilon_C$, and the rating floor $R(O_G) \geq \epsilon_R$. Applying this ILP to multiple threshold pairs and filtering the assignments by dominance produces a sampled frontier. A suitable threshold pair can recover an unsupported frontier point because the point need not maximize a weighted combination of the objectives.

With g thresholds between 0 and αB for ϵ_C and g thresholds over the candidate rating range for ϵ_R , the Cartesian product contains g^2 pairs. Each pair requires a separate ILP run. A coarse grid can miss large portions of the frontier, while a fine grid increases the number of ILP runs. We therefore explore methods that construct multiple assignments incrementally.

3.2 PBS: Pareto Beam Search

Pareto Beam Search (PBS) bounds the partial assignments retained by Pareto dynamic programming [3, 19, 23]. Without beam truncation, it computes exact frontiers using candidate reduction and dominance based on raw cost. We start with an empty assignment and process the requirements in order. After processing the first i requirements, a partial assignment selects one candidate for each of $q_1, \dots, q_i$. To process the next requirement, PBS extends every partial assignment with each candidate for that requirement and retains at most b resulting partial assignments that represent different tradeoffs. We discuss details below.

3.2.1 Beam Construction. To ease presentation, we call a partial assignment *feasible* if every selected item belongs to its requirement's candidate set and its raw accumulated cost does not exceed the relaxed budget cap. PBS retains only feasible extensions. If none remains, it returns an empty result. A feasible completion is a complete feasible assignment as defined in Definition 2.2.

Before constructing the beam, we retrieve a candidate set for each requirement and apply three dimensional candidate reduction once. The reduction removes a candidate if another candidate for the same requirement has no lower matching quality or rating and no higher cost. This replacement preserves feasibility and cannot worsen any Pareto objective. The reduction removes a candidate only if it is strictly dominated in matching quality, cost, and rating, or if it is identical in all three values to a retained candidate, of which exactly one is kept. Every removed candidate is therefore weakly dominated by a retained candidate.

3.2.2 Partial Assignment Pruning and Beam Retention. After generating feasible extensions, we then apply partial assignment dominance to prune partial assignments. Note that such a step is performed separately after the three dimensional candidate reduction.

The pruning step must determine which partial assignments can be removed without losing a possible Pareto optimal completion. The Pareto CGQ objective $(Q, -\Delta C, R)$ is unsafe for this comparison. Every partial assignment whose cost is below B has zero cost overhead, even though partial assignments with different raw accumulated costs have different remaining budgets.

We instead compare partial assignments using $(Q, -C, R)$. For a partial assignment v , let C_v , Q_v , and R_v denote its raw accumulated

cost, solution quality, and total rating. Suppose a partial assignment v' has no lower quality or rating than v and costs no more. Any suffix feasible for v can also be appended to v' . The resulting completion from v' remains feasible and has an objective vector no worse than the corresponding completion from v . We call this comparison partial assignment dominance.

LEMMA 3.1 (PARTIAL ASSIGNMENT DOMINANCE). *Let v and v' be partial assignments at the same depth with the same remaining candidate sets. If $Q_{v'} \geq Q_v$, $C_{v'} \leq C_v$, and $R_{v'} \geq R_v$, then the suffix of any feasible completion of v can be appended to v' to produce a feasible assignment whose objective vector is at least as good under $(Q, -\Delta C, R)$.*

PROOF SKETCH. Let σ be the suffix of any feasible completion of v . The same suffix is available to v' , and $C_{v'} \leq C_v$ ensures that appending it remains feasible. It adds the same contributions to both partial assignments, so the completion from v' is at least as good in every objective. $\square$

An instance with two requirements illustrates why cost overhead is unsafe before completion. Let $B = 10$, let $\alpha = \beta = 0$, and set every rating to zero. The first requirement has candidates a with quality 0.8 and cost 0, and a' with quality 0.9 and cost 10. The second has candidates b with quality 0 and cost 0, and b' with quality 1 and cost 10. After the first requirement, both partial assignments have zero cost overhead, so (a') dominates (a) under $(Q, -\Delta C, R)$. Removing (a) leaves only the feasible completion (a', b) with quality 0.45. However, (a, b') is feasible with quality 0.9 and dominates (a', b) . A larger beam width cannot recover (a, b') once (a) is removed, while raw cost keeps the partial assignments incomparable.

The remaining partial assignments can be mutually incomparable under $(Q, -C, R)$. If more than b partial assignments remain, we retain the top b according to the scalar proxy F' . For a partial assignment v , we evaluate the weighted objective in Equation (1) using Q_v , $\max\{C_v - B, 0\}$, and R_v , with weights chosen for search guidance. We use F' only for beam truncation. It does not define dominance or the Pareto CGQ objective. After all requirements are assigned, we return one representative assignment for each nondominated objective vector under $(Q, -\Delta C, R)$.

3.2.3 The PBS Algorithm. Algorithm 1 summarizes the complete procedure. We first retrieve and reduce the candidate sets and initialize the beam with the empty partial assignment (Lines 1–2). At each search level, PBS generates feasible extensions and removes extensions dominated under $(Q, -C, R)$. An extension is feasible when its cost plus the minimum total cost of selecting one candidate for each remaining requirement does not exceed $(1 + \alpha)B$. This test discards partial assignments that cannot be completed within the relaxed budget cap. We implement dominance filtering as a three dimensional maxima computation [20]. PBS then applies beam truncation only when more than b incomparable partial assignments remain (Lines 3–12). It then constructs P from the complete assignments in the final beam (Lines 13–17). `UpdateFrontier` inserts an assignment and removes the objective vectors it dominates under $(Q, -\Delta C, R)$, retaining one representative for each vector.

3.2.4 Guarantees and Complexity. Every extension satisfies the per-item matching constraint and the relaxed budget cap. Therefore,

Algorithm 1 Pareto Beam Search (PBS)

Require: CGQ instance (G, B, α, β) and beam width b
Ensure: Frontier P over the complete assignments retained by the beam

```
1:  $C_i \leftarrow \text{Reduce3D}(\text{retrieve}(q_i, (1 + \alpha)B, \beta))$  for every  $q_i$ 
2:  $\mathcal{B} \leftarrow \{\emptyset\}$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $\mathcal{B}' \leftarrow \text{FeasibleExtensions}(\mathcal{B}, C_i, (1 + \alpha)B)$ 
5:   if  $\mathcal{B}' = \emptyset$  then return  $\emptyset$ 
6:   end if
7:    $\mathcal{B}' \leftarrow \text{nondominated under } (Q, -C, R)$ 
8:   if  $|\mathcal{B}'| > b$  then
9:      $\mathcal{B}' \leftarrow \text{top } b \text{ by } F'(\cdot)$ 
10:  end if
11:   $\mathcal{B} \leftarrow \mathcal{B}'$ 
12: end for
13:  $P \leftarrow \emptyset$ 
14: for  $s \in \mathcal{B}$  do
15:    $\text{UpdateFrontier}(P, s)$ 
16: end for
17: return  $P$ 
```

every assignment returned by PBS is feasible for any beam width. Every retained partial assignment also has a feasible completion. For a feasible instance and $b \geq 1$, adding the least costly candidate for the next requirement passes the extension test, and dominance filtering and beam truncation retain at least one extension. Thus, PBS returns at least one assignment if it finishes all search levels. We show that without beam truncation, PBS also preserves a partial assignment that leads to each Pareto optimal objective vector.

THEOREM 3.2 (PARETO COMPLETENESS OF PBS). *PBS without beam truncation returns a representative of every Pareto optimal objective vector over the supplied candidate sets when it applies partial assignment dominance under $(Q, -C, R)$.*

PROOF SKETCH. Without beam truncation, each search level retains, for every Pareto optimal objective vector, a partial assignment that can complete to it. This holds initially for the empty assignment. If an extension is removed by dominance filtering, Lemma 3.1 ensures that a retained dominating assignment has a completion at least as good. Since the target vector is Pareto optimal, that completion must have the same vector. Induction over the requirements therefore preserves every frontier vector through the final filter. Candidate reduction extends the guarantee to the supplied candidate sets. $\square$

Complexity of PBS. Let N be the total number of retrieved candidates and m the largest reduced candidate set size. PBS evaluates at most nbm extensions. With three dimensional maxima filtering, candidate reduction, beam sorting, and explicit assignment copying, its time after retrieval is $O(N \log(N + 1) + nbm(\log(bm + 1) + n))$. A materialized expansion contains at most bm partial assignments, each storing up to n item identifiers, giving $O(bmn)$ space excluding candidate sets.³ Bounded PBS returns at most b frontier points. On the instances of Theorem 2.4, its recall is therefore at most $b/2^n$, and complete recovery requires $b \geq 2^n$.

³The PBS proof and complexity details appear in the technical report [1].

PBS is useful when bounding the retained search state is important. A bounded beam limits the number of partial assignments retained at every search level, but truncation is irreversible. A useful partial assignment may be removed when its current proxy score is low or when more than b partial assignments are mutually incomparable. Increasing b trades additional time and memory for greater frontier recovery, while removing the width limit gives the complete frontier by Theorem 3.2.

3.3 PTS: Pareto Tree Search

PBS bounds the partial assignments retained at each search level using a beam of width b . This provides direct control over the retained search state, but a partial assignment removed by beam truncation cannot be recovered later. To retain alternatives across iterations, we introduce a tree search method that organizes partial assignments according to their extensions. Each node represents a partial assignment, and each outgoing edge selects a candidate for the next requirement. We iteratively expand and search this tree and call the resulting method Pareto Tree Search (PTS).

3.3.1 Tree Construction. To construct the tree for search, we use root r to store the empty partial assignment at depth zero. Each depth corresponds to one requirement, so a terminal node at depth $n = |G|$ stores a complete assignment. Each node stores its partial assignment, its children, and the candidates not yet processed.

Before constructing the tree, we retrieve a candidate set for each requirement and apply the same three dimensional candidate reduction used by PBS. This reduction preserves every distinct Pareto optimal objective vector. The resulting candidate sets remain fixed throughout the search, and their Cartesian product defines the reduced search space.

In each iteration, we select a node with remaining search work and create at most one child by assigning a candidate for the next requirement. The original partial assignment remains in the tree, so a later iteration can return to it and create another child. When we reach a terminal node, we use its complete assignment to update the current frontier P .

3.3.2 Pruning Rules. During tree search, we prune a subtree only when none of its feasible completions can contribute a new objective vector. We introduce three rules: (1) The *reachability bound* determines whether a feasible completion exists. (2) The *objective upper bound* determines whether any completion can improve the current frontier. (3) *Partial assignment dominance* determines whether another partial assignment can represent all completions of the current partial assignment.

To ease presentation, for a node v , let d_v denote its depth and let $\mathcal{T}(v)$ denote its feasible completions in the fixed reduced search space. For each index $d \in \{1, \dots, n\}$, we define three statistics over the remaining requirements $q_d, \dots, q_n$. The value $\min C[d]$ is the minimum total cost of selecting one candidate for each requirement. The values $\max Q[d]$ and $\max R[d]$ sum the maximum solution quality and total rating contributions, respectively. We set all three statistics to zero at $d = n + 1$. If any candidate set for requirements $q_d, \dots, q_n$ is empty, $\min C[d]$ is infinity.

Reachability bound. We first test whether a feasible completion exists. A feasible completion of v can exist only if

$$C_v + \min C[d_v + 1] \leq (1 + \alpha)B. \quad (2)$$

The first term is the accumulated cost of the current partial assignment. The second term is the least additional cost required to assign all remaining requirements. Every retained candidate already satisfies the matching threshold, and selecting the least costly candidate independently for each remaining requirement achieves this lower bound. Thus, the left side is the minimum total cost of any completion of v . Equation (2) fails exactly when $\mathcal{T}(v)$ is empty, in which case we can prune v directly.

Objective upper bound. We next test whether any feasible completion can improve the current frontier. These statistics give the following upper bound on the objective vector

$$\begin{aligned} \text{ub}(v) = & (Q_v + \max Q[d_v + 1], \\ & - \max\{0, C_v + \min C[d_v + 1] - B\}, \\ & R_v + \max R[d_v + 1]). \end{aligned} \quad (3)$$

The first and third components bound solution quality and total rating by adding their accumulated values to the maximum contributions from the remaining requirements. The second uses the minimum possible total cost to bound negative cost overhead. These component bounds may come from different completions, so $\text{ub}(v)$ need not represent one feasible assignment. However, it weakly dominates the objective vector of every completion of v . If an objective vector in P weakly dominates $\text{ub}(v)$, it also weakly dominates the objective vector of every assignment in $\mathcal{T}(v)$.

Partial assignment dominance. At each depth d , we maintain a partial assignment archive $D[d]$ containing the partial assignments retained under $(Q, -C, R)$. We apply the same comparison as in Section 3.2.2. Since partial assignments at the same depth have the same remaining candidate sets, Lemma 3.1 applies directly.

The archive maintains a representative for every partial assignment removed by this rule. A representative remains available for exploration or belongs to an exhausted branch whose completions have already been covered. If a representative is later dominated, we transfer this relation to its dominating partial assignment. Tied partial assignments use one canonical representative and need to be explored only once, so an extension whose comparison values equal those of an archived entry is not created. The archive satisfies two invariants that the soundness proof uses. Every entry of $D[d]$ is a partial assignment that has been created as a tree node, and an entry is replaced only by a created partial assignment that weakly dominates it under $(Q, -C, R)$.

Before creating a child, we check whether an entry in the archive weakly dominates its partial assignment. This check does not modify the archive. If the extension survives pruning, we create the child and link it to its parent before inserting it into the archive. We remove any archive entries weakly dominated by the new child, but their tree nodes remain available until exhausted.

THEOREM 3.3 (SOUNDNESS OF PTS PRUNING). *Let v be a partial assignment in the fixed reduced search space. Pruning v preserves a representative of every Pareto optimal objective vector if Equation (2) fails, if an objective vector in P weakly dominates $\text{ub}(v)$, or if a partial assignment in $D[d_v]$ weakly dominates v under $(Q, -C, R)$.*

PROOF SKETCH. Failure of Equation (2) leaves no feasible completion. A frontier vector weakly dominating $\text{ub}(v)$ also weakly dominates every completion of v . Lemma 3.1 guarantees corresponding completions at least as good from an archive representative. Later updates retain dominating vectors and keep representative branches available until their completions are covered. Hence, all three rules preserve every Pareto optimal objective vector. $\square$

3.3.3 The PTS Algorithm. The pruning rules determine which branches can be skipped. To explore the remaining branches, we use depth first search (DFS), ordering feasible children by the scalar proxy F' introduced in Section 3.2.2. DFS explores each child's subtree before proceeding to its siblings and considers the next untried candidate when no unexhausted child remains.

After expanding a child, we greedily complete its partial assignment using F' . If greedy completion fails even though Equation (2) holds, we select the least costly candidate for every remaining requirement. The resulting assignment updates the frontier P .

Algorithm 2 shows the full algorithm. We introduce an iteration budget M to limit the number of search iterations. PTS first performs the same candidate retrieval and reduction as PBS, then initializes the tree and its search state (lines 1–2). A terminal node is exhausted after its assignment is evaluated (line 13). An internal node is exhausted only when its candidate list has been initialized, every candidate has been expanded or pruned, and every child is exhausted. An uninitialized list does not establish exhaustion. Each of at most M iterations selects an unexhausted node in depth first order and considers candidates in F' order until it finds an extension that is not removed by the three pruning rules (lines 6–7). Selection initializes each candidate list when first needed. If no extension remains, including when the initialized list is empty, PTS checks exhaustion at the selected node and propagates it toward the root (line 9). A node with an unexhausted child remains available for later selection. When an extension survives, PTS obtains a feasible completion, updates the current frontier using the same UpdateFrontier operation as PBS (lines 11–12), and records any newly exhausted nodes (lines 13–14). The search returns the current frontier when the iteration budget is reached or the root is exhausted (lines 3–4 and 16).

If the iteration budget ends before the root is exhausted, PTS returns the current frontier P . PTS without the candidate cap retains every candidate in the reduced candidate sets, so at root exhaustion, P is the complete frontier over the original candidate sets.

THEOREM 3.4 (PARETO COMPLETENESS OF PTS). *Let $\mathcal{T}$ be the feasible assignments formed from the fixed reduced candidate sets, let $|V| = \sum_{d=0}^n \prod_{i \leq d} |C_i|$ be the number of nodes of the fixed reduced tree. Assume exact arithmetic. At every iteration, P contains the non-dominated objective vectors of the complete assignments evaluated so far. Continued search exhausts the root after at most $2|V|$ iterations. At root exhaustion, P is the complete frontier over $\mathcal{T}$. Candidate reduction preserves this frontier over the original candidate sets.*

PROOF SKETCH. Frontier updates maintain exactly the nondominated vectors of the assignments evaluated so far. Each iteration either creates a child or finishes a node's candidate list without creating a child. Each event occurs at most once per node, so continued DFS exhausts the root within $2|V|$ iterations. At exhaustion,

Algorithm 2 Pareto Tree Search (PTS)

Require: CGQ instance (G, B, α, β) , iteration budget M **Ensure:** Current frontier P over the complete assignments evaluated so far

```
1:  $C_i \leftarrow \text{Reduce3D}(\text{retrieve}(q_i, (1 + \alpha)B, \beta))$  for every  $q_i$ 
2: Initialize root  $r$ , frontier  $P \leftarrow \emptyset$ , archives  $D$ , and suffix bounds
3: for  $t \leftarrow 1$  to  $M$  do
4:   if  $r$  is exhausted then break
5:   end if
6:    $v \leftarrow \text{SelectUnexhausted}(r)$  ▷ depth first search
7:    $u \leftarrow \text{NextUnprunedExtension}(v, P, D, F')$  ▷ apply pruning rules and  $F'$  ordering
8:   if  $u = \text{null}$  then
9:      $\text{PropagateExhaustion}(v)$ , then continue
10:  end if
11:   $s \leftarrow \text{CompleteFeasible}(u, F')$  ▷ least cost completion as fallback
12:   $\text{UpdateFrontier}(P, s)$ 
13:  Mark  $u$  evaluated if it is a terminal node
14:   $\text{PropagateExhaustion}(u)$ 
15: end for
16: return  $P$ 
```

every feasible assignment has been evaluated or covered by sound pruning under Theorem 3.3. Thus, P contains the complete frontier. $\square$

Complexity Analysis. Let $m = \max_i |C_i|$, $|P|$ be the largest current frontier size, and $D = \max_d |D[d]|$. Including traversal, candidate sorting, assignment copying, completion, and dominance checks, M iterations take $O(M(nm + m \log(m+1) + m(|P| + D) + |P| + n))$ time. After candidate reduction, initialization costs $O(nm \log(m+1))$, while final filtering and assignment reconstruction cost $O(|P| \log(|P| + 1) + n|P|)$. Retrieval and candidate reduction are separate. At most one node is created per iteration. Including stored assignments, candidate lists, archives, and the frontier, space is $O(M(n + m) + nm + nD + |P|)$, excluding the original retrieved candidate lists.⁴

3.3.4 Optional Candidate Cap. Even with three dimensional candidate reduction, a large candidate set can still make the PTS tree expensive to explore. As an optional optimization, we apply a candidate cap that retains only the first κ candidates. Candidates $o \in C_i$ are ranked by decreasing $\lambda_1 f_m(q_i, o)/n + \lambda_3 r_o/R_{\max}$. The cap replaces C_i with $\widehat{C}_i = \text{top}_\kappa(C_i)$ and reduces the branching of the tree. A candidate outside the cap may be needed for the complete frontier, so capped PTS is not Pareto complete for the original candidate sets. The capped search space is $\widehat{\Omega} = \prod_i \widehat{C}_i$. With the cap enabled, the search terms use $\min\{m, \kappa\}$ in place of m . Candidate reduction, initial ordering, and storage of the full reduced candidate sets still depend on their uncapped sizes.

4 EXPERIMENTS

4.1 Experimental Setup

Dataset. We use two datasets from different domains, Amazon Reviews’23 [14] and Inside Airbnb listings⁵. Amazon Reviews’23 contains 48.19M items across 33 categories. We sample 10,000 items

from each of 11 categories, which produces the 110K item catalog used in the default experiments. We also construct larger Amazon catalogs with up to 2.14M items for the scalability study. Inside Airbnb covers 12 cities and contains 230,926 listings after cleaning, with 4.8K to 40.3K listings per city. We process the title text from both datasets before embedding. For Amazon, we extract keywords from the original titles and use Qwen2.5-7B-Instruct [34] to generate cleaned titles. We use the same model to clean the Airbnb listing names. We embed all cleaned titles with Qwen3-Embedding-0.6B [35]. For Airbnb, we also convert prices to USD using fixed exchange rates and define the rating as the mean of its six review subscores.

Workload. We evaluate 100 queries for each dataset. For Amazon, we generate natural language requirements from product keywords and combine them across categories. All queries are generated to be feasible at $\alpha = 0$. Queries contain 3 to 8 requirements with a median of 5. At $\beta = 0.8$, retrieval returns a median of 160 candidates per query. For 41 queries, the cheapest feasible assignment consumes more than 80% of the target budget. For Airbnb, we generate templated stay requirements from listing attributes for distinct cities, selecting one listing per city. Queries contain 3 to 8 requirements with a median of 6. At $\beta = 0.8$, retrieval returns a median of 1,726 candidates per query. Of these queries, 61 have an assignment within the target budget, and all 100 are feasible at the default $\alpha = 0.5$ and $\beta = 0.8$. Workload generation and selection details appear in the technical report [1].

Methods and default parameters. We use PBS and PTS for Pareto CGQ and compare them with Pareto dynamic programming (DP), repeated ILP, and exhaustive enumeration. We separately evaluate weighted PBS and weighted PTS for Weighted CGQ. Each method has a 60 second time limit per query. **(1) PBS.** PBS constructs assignments one requirement at a time and retains at most b partial assignments at each search level. We also compare PBS without beam truncation. **(2) PTS.** PTS explores partial assignments in a tree until the root is exhausted or it reaches the iteration budget M . **(3) DP.** DP keeps partial assignments nondominated on quality, raw cost, and rating after each requirement. It uses the retrieved candidates without candidate reduction. **(4) Repeated ILP (ϵ -ILP).** The ϵ -ILP baseline maximizes solution quality while imposing bounds on cost overhead and total rating. We vary both bounds over a grid and solve one ILP for each pair. We show $g = 4$ and $g = 16$. **(5) Pareto brute force (PBF).** PBF exhaustively enumerates feasible assignments and retains the nondominated objective vectors. Partial frontiers returned before a timeout are retained. Unless otherwise stated, we use $\alpha = 0.5$, $\beta = 0.8$, $b = 100$, $M = 500$, and $\kappa = 50$ on both datasets. The search guidance weights are $\lambda_1 = \lambda_2 = \lambda_3 = 1$.

Metrics. We define each query’s reference frontier as the complete frontier returned by PBS without beam truncation over the retrieved candidate sets (Theorem 3.2). **Latency** includes retrieval, candidate reduction, search, and output preparation. Query embeddings are precomputed. We use a p90 latency target of 100 ms for candidate retrieval and CGQ processing as a design assumption for interactive use [9]. **Frontier recall** is the fraction of reference frontier points returned by a method. **Hypervolume ratio (HV)** is the dominated volume of the returned frontier divided by that of the reference frontier. Both volumes use a common reference point set 10^{-3} below

⁴Full PTS proofs and complexity details appear in the technical report [1].

⁵<https://insideairbnb.com>, CC BY 4.0.

each coordinate minimum of the complete reference frontier, using summed matching quality, negative cost overhead, and total rating. We report both metrics because HV can remain high when a method recovers central tradeoffs but misses other frontier points. **Success rate** is the fraction of queries for which a method returns at least one feasible assignment. We report success rate for comparisons in which it varies. Recall and HV are averaged over feasible queries with a complete reference frontier. If a method finds no assignment, both metrics are set to zero and the query remains in the average. All 100 queries per dataset are included at the default settings.

Implementation. All methods are implemented in Python and evaluated on a server with two Intel Xeon Platinum 8452Y CPUs and two NVIDIA RTX 5000 Ada GPUs. Retrieval scans the full catalog using the matching threshold and relaxed item price cap. Retrieval timing includes query transfer and candidate transfer to the CPU, catalog loading and embedding preparation are excluded. The ϵ -ILP baseline uses SciPy’s `milp` interface to HiGHS. Weighted CGQ uses our branch and bound solver for the ILP in Section 3.1.1, described in the technical report [1].

4.2 Experimental Results

Our experiments address four research questions.⁶ **RQ1.** How do PBS and PTS compare with the baselines in frontier recall and end to end latency? **RQ2.** How do the search parameters and algorithm components affect frontier recall and latency? **RQ3.** How do catalog size and query length affect frontier recall and latency? **RQ4.** How do weighted PBS and weighted PTS compare with the exact ILP when preference weights are known?

4.2.1 RQ1. Frontier Recovery and Efficiency. We compare all methods at the default setting. Figure 2 shows the per query recall and latency distributions for Amazon and Airbnb, and Tables 1 and 2 summarize the results. The reference frontier contains a median of 55 points per Amazon query and 248 points per Airbnb query.

Default comparison. On Amazon, PBS and PTS have higher mean recall and lower median latency than the densest ϵ -ILP grid, although the grid has higher HV. PBF has higher mean recall than PBS and PTS, but its median latency is 1.18 s. Every method returns an assignment for every query. PBS without beam truncation and DP recover the complete frontier at medians of 2.9 and 5.3 ms. PBS without beam truncation applies candidate reduction before search, while DP processes the retrieved candidates without reduction.

On Airbnb, PBS has higher mean recall than PTS (0.456 versus 0.392) and slightly lower median latency (14.0 versus 14.3 ms). PBS without beam truncation recovers the complete frontier at a median latency of 20.9 ms, and DP recovers it on 99 of 100 queries at 167.6 ms, reaching the 60 second limit on the query with the largest frontier. PBS and PTS meet the assumed p90 target at 36.4 and 26.3 ms, while PBS without beam truncation exceeds it at 229.4 ms. Bounded search reduces tail latency at the cost of frontier recovery. The densest ϵ constraint grid again has higher HV despite lower recall. Airbnb’s median frontier is about four times Amazon’s, with up to 3,042 points. PBF completes 75 of 100 Amazon queries and 21 of 100 Airbnb queries, with all remaining runs reaching the time limit. On Airbnb, its median latency is 60.01 s, and 49 runs return

Table 1: Default frontier results on Amazon.

method	recall	HV	success	runs	median	p90
PBS ($b=100$)	0.727	0.930	1.00	1	2.9 ms	8.3 ms
PTS ($M=500$)	0.784	0.926	1.00	1	7.0 ms	11.3 ms
PBS unbounded	1.000	1.000	1.00	1	2.9 ms	12.1 ms
DP	1.000	1.000	1.00	1	5.3 ms	69.1 ms
ϵ -ILP $g=4$	0.136	0.582	1.00	17	129.4 ms	272.8 ms
ϵ -ILP $g=16$	0.465	0.954	1.00	257	2.02 s	4.26 s
PBF	0.844	0.929	1.00	1	1.18 s	60.23 s

Table 2: Default frontier results on Airbnb.

method	recall	HV	success	runs	median	p90
PBS ($b=100$)	0.456	0.820	1.00	1	14.0 ms	36.4 ms
PTS ($M=500$)	0.392	0.643	1.00	1	14.3 ms	26.3 ms
PBS unbounded	1.000	1.000	1.00	1	20.9 ms	229.4 ms
DP	0.990	0.990	0.99	1	167.6 ms	2.48 s
ϵ -ILP $g=4$	0.045	0.432	1.00	17	416.0 ms	1.34 s
ϵ -ILP $g=16$	0.178	0.878	1.00	257	7.09 s	20.21 s
PBF	0.253	0.358	0.51	1	60.01 s	60.19 s

no assignment. Every completed PBF frontier matches PBS without beam truncation.

Effect of search budgets. We vary the beam width b for PBS and the iteration budget M for PTS in Figures 3a and 3b. These parameters control different search operations and do not define equal amounts of computation. On Amazon, the median latency of PBS ranges from 1.1 to 3.0 ms, and PBS recovers the complete frontier at the largest beam width. The median latency of PTS ranges from 4 to 7 ms, and its mean recall reaches 0.992 at $M = 8000$. On Airbnb, PBS and PTS require more time and recover less of the larger frontiers within the tested budgets. Increasing b or M raises frontier recall on both datasets. We select the defaults $b = 100$ and $M = 500$ using the Amazon results and apply them unchanged to Airbnb.

Frontier availability during search. We measure frontier recall and HV at common elapsed times on Amazon in Figure 4. Each method runs in a separate process. Times exclude retrieval. The plots sample one invocation at successive elapsed times.

PTS returns its first frontier point in 1.6 ms at the median, compared with 5.2 ms for PBS, and reaches median recall 1.000 by 300 ms. PBS returns all retained points after beam construction, when its recall is higher. At one second, PTS has higher mean recall than PBS (0.784 versus 0.727), but slightly lower mean HV (0.926 versus 0.930). For ϵ -ILP, the $g = 16$ grid attains mean HV 0.539 at this elapsed time. Its higher HV in the default comparison does not imply higher HV at the same elapsed time.

Takeaway. PBS without beam truncation has comparable median latency. On Airbnb, bounded PBS and PTS meet the assumed tail latency target, which PBS without beam truncation exceeds. PTS provides frontier points during search. PBS and PTS return assignments for all 100 queries on each dataset.

4.2.2 RQ2. Parameter Sensitivity and Component Ablation. *Parameter sensitivity.* On Amazon, PBS median recall increases from 0

⁶Additional experimental results are available in the technical report [1].

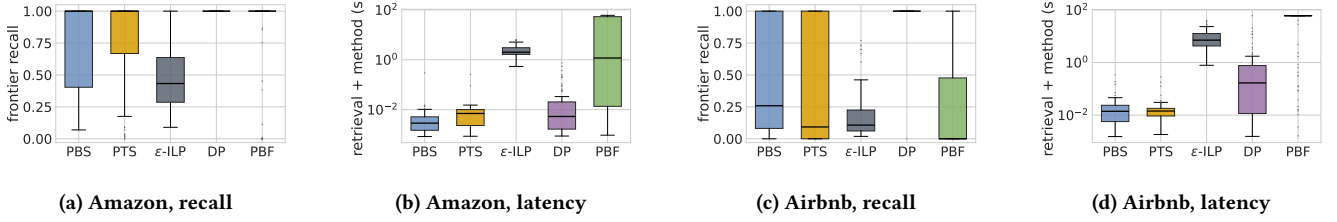


Figure 2: Default frontier recall and latency. Distributions include all 100 queries per dataset for every method.

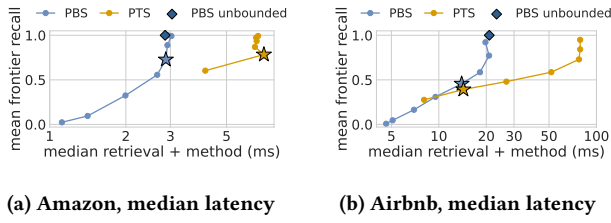


Figure 3: Frontier recall versus median latency as b and M vary. Stars mark defaults and diamonds unbounded PBS.

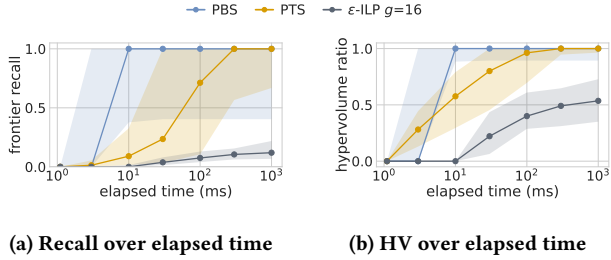


Figure 4: Frontier availability during search on Amazon. Times exclude retrieval.

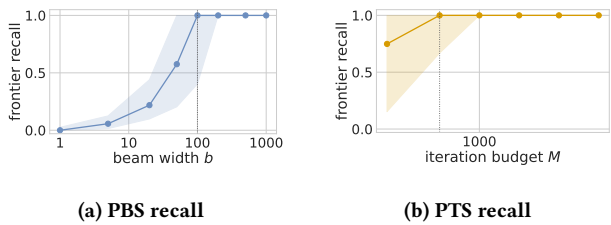


Figure 5: Effect of b and M on frontier recall on Amazon. Medians with interquartile ranges.

at $b = 1$ to 1 at $b = 100$, with median latency below 3.2 ms (Figure 5). Larger beams increase the upper quartile of latency with little change in median recall, as detailed in the technical report [1]. PTS reaches median recall 1 at $M = 500$, exhausting the root for 68% of queries. Its median latency stays near 6.6 ms for larger M . *Budget relaxation.* A larger α permits more assignments above the target budget and reduces the normalized cost overhead of a fixed assignment. Using the same 100 Amazon queries and their stored

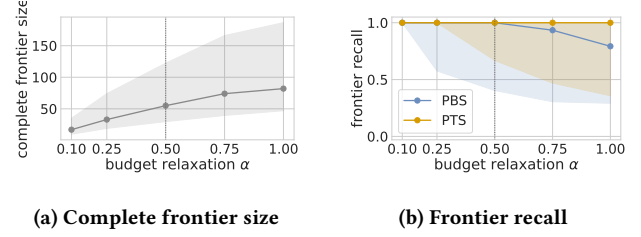


Figure 6: Budget relaxation on Amazon.

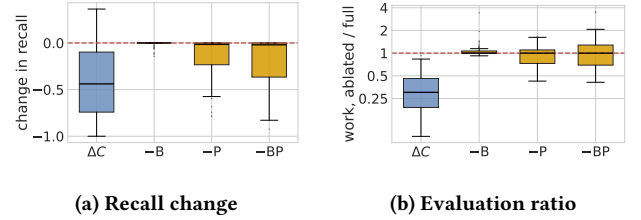


Figure 7: Component ablations on Amazon. ΔC uses cost overhead for PBS. For PTS, $-B$ removes the objective upper bound, $-P$ removes partial assignment dominance, and $-BP$ removes both rules.

budgets, we vary α from 0.1 to 1.0 in Figure 6. At each setting, we retrieve candidates under the relaxed price cap and recompute the reference frontier. All queries remain feasible and have complete reference frontiers. At the default beam width and iteration budget, the median complete frontier grows from 17 to 82 points. Frontier recall decreases as the fixed search budgets cover a smaller fraction of these frontiers, while the mean hypervolume ratio remains at least 0.88 for PBS and PTS.

Component ablation. We evaluate four ablations on Amazon in Figure 7. Candidate evaluations are compared only within each method.

Replacing raw cost with cost overhead reduces PBS recall on 85% of queries, with a median loss of 0.439. Below the target budget, equal cost overhead hides differences in remaining budgets, allowing useful partial assignments to be discarded. Candidate evaluations decrease, but so does frontier recall, consistent with Lemma 3.1. For PTS, removing partial assignment dominance reduces recall on 53 queries and lowers mean recall from 0.784 to 0.640. Removing the objective upper bound reduces recall on 15 queries, with mean recall 0.774. Removing both rules lowers mean

recall to 0.601. The median paired candidate-evaluation ratio is 1 for all three ablations under the fixed iteration budget.

Candidate cap. At $\kappa = 50$, PTS matches uncapped median frontier recall on both datasets. On Amazon, every returned frontier and the median candidate evaluations of 1,795 remain unchanged. Additional results for κ and β appear in the technical report [1].

Takeaway. Raw-cost dominance in PBS and partial assignment dominance in PTS improve frontier recall. For PTS, these gains do not reduce median candidate evaluations under the iteration budget.

4.2.3 RQ3. Scalability. Catalog size. We vary the Amazon catalog from 10K to 30K items per category in Figure 8. The same 100 queries retain their target budgets from the 110K catalog. At 30K items per category, median recall is 0.29 for PBS and 0.77 for PTS, and latency increases for both methods. With fixed search budgets, the methods examine a smaller fraction of the candidate combinations as the catalog grows.

Query length. On Amazon, increasing the query length from 2 to 8 requirements reduces median recall to 0.17 for PBS and 0.02 for PTS, while latency increases for both methods. Each additional requirement adds a search level and increases the number of candidate combinations. The fixed beam width and iteration budget therefore cover a smaller fraction of the search space. Airbnb shows similar trends, with detailed results in the technical report [1].

We increase each method’s search budget across the Amazon query lengths in Figure 9. At length 8, PBS reaches median recall 1.0 with $b = 800$ in approximately 22 ms, while PTS reaches median recall 1.0 with $M = 8000$ in approximately 131 ms. Their mean recalls are 0.913 and 0.868, respectively. Increasing the search budgets improves recall over the default parameters.

With eight requirements on Airbnb, PBS and PTS have p90 latencies of 41 and 29 ms, compared with 419 ms for PBS without beam truncation, as shown in Figure 10. Their sampled memory increases are 1.3 and 4.0 MB, compared with 16.5 MB for PBS without beam truncation. Both methods meet the assumed latency target, with mean frontier recalls of 0.170 and 0.053, respectively.

Million item catalogs. We extend the Amazon catalog to 110K, 330K, 1.1M, and 2.14M items. At each catalog size, we recompute each query’s target budget as its fixed budget multiplier times the minimum feasible cost in that catalog. This keeps the fraction of tight budget queries at 0.41 and isolates the effect of catalog size [1]. Candidate retrieval accesses the catalog, while PBS and PTS operate on the retrieved candidate sets. Catalog growth therefore affects the search through changes in candidate set size.

Figure 11a reports latency over these catalogs. PBS median latency increases from 2.9 to 20 ms, while PTS increases from 7.1 to 21 ms. The shared retrieval stage reaches 11 ms at 2.14M items. PBS without beam truncation and DP recover the complete frontier on all 100 queries at each Amazon catalog size. At 2.14M items, their median latencies are 20.7 and 70.6 ms, respectively, while DP’s 90th percentile latency is 733 ms, much higher than 49 ms for PBS without beam truncation. The technical report includes its latency and the retrieval timing on both datasets [1]. Measurements for ϵ -ILP end at 330K items and PBF at 110K items, where these methods already require seconds for many queries.

Figure 11b reports frontier recall over the same catalogs. At 2.14M items, median recall is 0.54 for PBS and 0.53 for PTS under the

Table 3: Weighted CGQ results. Optimal is the fraction of instances that attain the exact ILP objective value.

method	Amazon		Airbnb	
	optimal	med. time	optimal	med. time
ILP (exact)	1.000	1.17 ms	1.000	4.16 ms
PBS	1.000	1.28 ms	0.984	5.91 ms
PTS	1.000	1.35 ms	0.960	5.83 ms

default search budgets. As the number of candidate combinations grows, the fixed beam width and iteration budget cover a smaller fraction of the search space. At every scale, $\kappa = 50$ closely matches the frontier recall of uncapped PTS, so the candidate cap accounts for little of this decline. At 2.14M items, the 99th percentile latency is 69 ms for PTS and 0.11 seconds for the separately timed PBS without beam truncation.

Takeaway. With fixed search budgets, PBS and PTS recover less of the frontier as catalogs and queries grow. Across the tested Amazon catalog sizes, PBS without beam truncation recovers the complete frontier with lower median latency than DP.

4.2.4 RQ4. Weighted CGQ. We compare the exact ILP with weighted PBS and weighted PTS on both datasets, as shown in Table 3. Every method uses the same weighted objective, and the ILP result serves as the reference across seven representative weight vectors. Each query is evaluated with seven weight vectors, giving 700 instances per dataset. All instances are feasible and have certified optima, and every method returns an assignment on every instance.

On Amazon, the ILP, weighted PBS, and weighted PTS return the optimum on every instance, with zero relative gaps. All three methods have similar median latency. The exact ILP has the lowest median, at 1.17 ms. On Airbnb, the ILP returns the optimum on all 700 instances at a median of 4.16 ms. Weighted PBS and weighted PTS return the optimum on 98.4% and 96.0% of instances.

Takeaway. When preference weights are known, the exact ILP attains the optimum on every evaluated instance and has the lowest median latency on both datasets.

5 RELATED WORK

Package and bundle recommendation studies select sets of items jointly rather than rank items independently. Early work formulated package recommendation as a constrained combinatorial selection problem, while later work considered personalized top- k packages, complementary shopping bundles, and neural bundle recommendation from user interactions [8, 10, 29, 31–33, 36]. Text2Bundle also incorporates a natural language query into bundle generation [15]. These methods generally generate or rank a bundle as a whole using compatibility or preference signals. In contrast, CGQ assigns a distinct item to each item requirement while enforcing a matching-quality threshold and a shared relaxed budget cap. Package queries are more closely related because they optimize sets of tuples under collective constraints [5–7, 11, 24]. CGQ uses a search space partitioned by requirement, where each item requirement induces a candidate set and exactly one item is selected from each set. The matching-quality threshold β and budget-relaxation parameter α

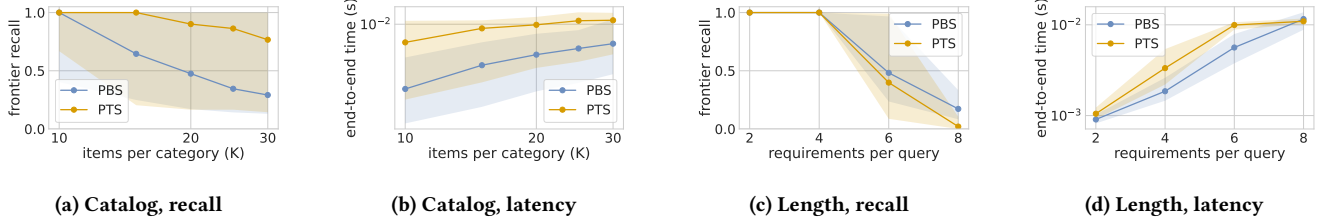


Figure 8: Catalog size and query length on Amazon. Medians with interquartile ranges.

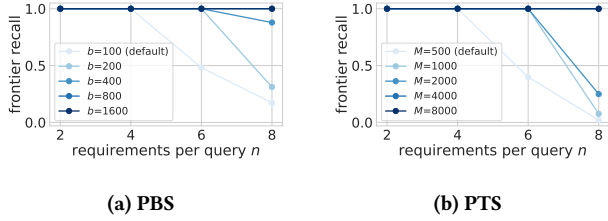


Figure 9: Frontier recall across query lengths and search budgets on Amazon.

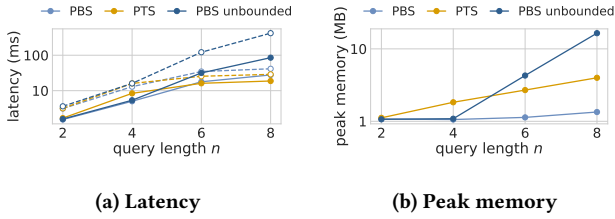


Figure 10: Latency and peak memory across query lengths on Airbnb. Solid and dashed lines show median and p90 latency. Memory is the maximum sampled increase after candidate loading over all queries.

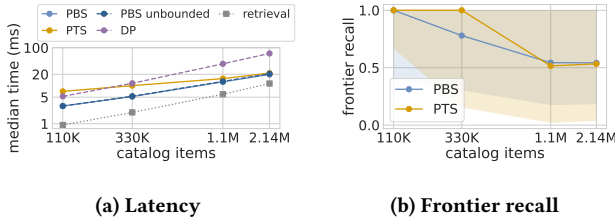


Figure 11: Scaling to 2.14M Amazon items. Budgets are recomputed at each catalog size.

connect semantic retrieval with item assignment. With eligible requirement and item pairs materialized as tuples, PaQL can express CGQ feasibility through requirement cardinality and aggregate budget constraints [6]. Recommendation under constraints also covers course planning with complex user requirements [27].

Pareto CGQ is related to multiobjective recommendation and Pareto query processing, where methods return tradeoffs rather

than a single weighted optimum [4, 17]. Weighted sum methods can miss unsupported frontier points, while the ϵ constraint method can recover such points through repeated optimization at predetermined settings. Our methods instead maintain nondominated alternatives during search. Liu et al. [21] define group based skyline for groups of k points using one to one point dominance. Pareto CGQ constructs assignments from a candidate set for each requirement under a shared budget and compares aggregate objectives.

CGQ is also related to the Multiple Choice Knapsack Problem, which selects one item from each group under a shared budget [18, 30]. This classical formulation optimizes a single objective, whereas Pareto CGQ returns nondominated tradeoffs among matching quality, budget overage, and rating. Klamroth and Wiecek [19] study multiobjective knapsack DP, while Bazgan et al. [3] prune partial solutions using dominance. Labeling methods for multiobjective and resource constrained shortest paths likewise prune partial paths using dominance over accumulated objectives and resource consumption [16, 25]. PBS adapts Pareto dynamic programming to requirement stages and limits retained assignments through beam truncation [23]. Partial assignments are compared using raw cost to preserve completion opportunities, while complete assignments use cost overhead. The DP baseline is this requirement-stage recurrence on the retrieved candidates without truncation and without candidate reduction. Florios et al. [12] apply multicriteria branch and bound to knapsack problems. PTS uses depth first search with partial assignment dominance and bounds over remaining requirements. Minimum completion cost bounds feasibility and cost overhead, while maximum quality and rating bound the other objectives.

Embedding based retrieval supplies the candidate set for each CGQ requirement [14, 35, 37]. Unlike methods that rank items independently, CGQ jointly selects one item per requirement under a shared budget.

6 CONCLUSION

Constraint Group Query selects one item per requirement under shared constraints. With known weights, Weighted CGQ returns an optimal assignment using ILP. When weights are unknown, Pareto CGQ computes complete frontiers using candidate reduction and partial assignment dominance based on raw cost. Bounded PBS limits retained assignments, while PTS maintains a frontier throughout tree search. Experiments on the Amazon and Airbnb datasets show that PBS without beam truncation constructs the complete frontier faster than DP and repeated ILP, while bounded PBS and PTS return partial frontiers under limited search time.

REFERENCES

- [1] Anonymous. 2026. GroupBuy: A Query Framework for Constrained Multi-Item Retrieval. Technical Report. <https://github.com/yestinchen/groupbuy/blob/main/report.pdf>. Available in the artifact repository.
- [2] Cristina Bazgan, Hadrien Hugot, and Daniel Vanderpooten. 2009. Implementing an Efficient FPTAS for the 0–1 Multi-objective Knapsack Problem. *European Journal of Operational Research* 198, 1 (2009), 47–56. <https://doi.org/10.1016/j.ejor.2008.07.047>
- [3] Cristina Bazgan, Hadrien Hugot, and Daniel Vanderpooten. 2009. Solving efficiently the 0–1 multi-objective knapsack problem. *Computers & Operations Research* 36, 1 (2009), 260–279. <https://doi.org/10.1016/j.cor.2007.09.009>
- [4] Stephan Börzsönyi, Donald Kossmann, and Konrad Stocker. 2001. The Skyline Operator. In *Proceedings of the 17th International Conference on Data Engineering*. 421–430. <https://doi.org/10.1109/ICDE.2001.914855>
- [5] Matteo Brucato, Azza Abouzied, and Alexandra Meliou. 2017. A Scalable Execution Engine for Package Queries. *SIGMOD Rec.* 46, 1 (May 2017), 24–31. <https://doi.org/10.1145/3093754.3093761>
- [6] Matteo Brucato, Azza Abouzied, and Alexandra Meliou. 2018. Package queries: efficient and scalable computation of high-order constraints. *The VLDB Journal* 27, 5 (2018), 693–718. <https://doi.org/10.1007/s00778-017-0483-4>
- [7] Matteo Brucato, Nishant Yadav, Azza Abouzied, Peter J. Haas, and Alexandra Meliou. 2020. Stochastic Package Queries in Probabilistic Databases. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (Portland, OR, USA) (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 269–283. <https://doi.org/10.1145/3318464.3389765>
- [8] Jianxin Chang, Chen Gao, Xiangnan He, Depeng Jin, and Yong Li. 2020. Bundle recommendation with graph convolutional networks. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1673–1676. <https://doi.org/10.1145/3397271.3401198>
- [9] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80. <https://doi.org/10.1145/2408776.2408794>
- [10] Ting Deng, Wenfei Fan, and Floris Geerts. 2012. On the Complexity of Package Recommendation Problems. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 261–272. <https://doi.org/10.1145/2213556.2213592>
- [11] Kevin Fernandes, Matteo Brucato, Rahul Ramakrishna, Azza Abouzied, and Alexandra Meliou. 2014. PackageBuilder: querying for packages of tuples. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. 1613–1614. <https://doi.org/10.1145/2588555.2612667>
- [12] Kostas Florios, George Mavrotas, and Danae Diakoulaki. 2010. Solving multiobjective, multiconstraint knapsack problems using mathematical programming and evolutionary algorithms. *European Journal of Operational Research* 203, 1 (2010), 14–21. <https://doi.org/10.1016/j.ejor.2009.06.024>
- [13] Zhankui He, Handong Zhao, Tong Yu, Sungchul Kim, Fan Du, and Julian McAuley. 2022. Bundle MCR: Towards Conversational Bundle Recommendation. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 288–298. <https://doi.org/10.1145/3523227.3546755>
- [14] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiushi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. *arXiv preprint arXiv:2403.03952* (2024).
- [15] Jialiang Hu, Chenyang Zhu, Yuwei Cui, Yinwei Shen, Rongrong Ji, and Zhongyu Wei. 2025. Text2Bundle: Towards Personalized Query-based Bundle Generation. *ACM Transactions on Recommender Systems* (2025). <https://doi.org/10.1145/3706125>
- [16] Stefan Irnich and Guy Desaulniers. 2005. Shortest Path Problems with Resource Constraints. In *Column Generation*, Guy Desaulniers, Jacques Desrosiers, and Marius M. Solomon (Eds.). Springer, 33–65. https://doi.org/10.1007/0-387-25486-2_2
- [17] Wen Jin, Martin Ester, Zengjian Hu, and Jiawei Han. 2007. The Multi-Relational Skyline Operator. In *Proceedings of the 23rd International Conference on Data Engineering*. 1276–1280. <https://doi.org/10.1109/ICDE.2007.368992>
- [18] Hans Kellerer, Ulrich Pferschy, and David Pisinger. 2004. *Knapsack Problems*. Springer, Berlin, Heidelberg. <https://doi.org/10.1007/978-3-540-24777-7>
- [19] Kathrin Klamroth and Margaret M. Wiecek. 2000. Dynamic programming approaches to the multiple criteria knapsack problem. *Naval Research Logistics* 47, 1 (2000), 57–76. [https://doi.org/10.1002/\(SICI\)1520-6750\(200002\)47:1<57::AID-NAV4>3.0.CO;2-4](https://doi.org/10.1002/(SICI)1520-6750(200002)47:1<57::AID-NAV4>3.0.CO;2-4)
- [20] H. T. Kung, F. Luccio, and F. P. Preparata. 1975. On Finding the Maxima of a Set of Vectors. *J. ACM* 22, 4 (1975), 469–476. <https://doi.org/10.1145/321906.321910>
- [21] Jinfei Liu, Li Xiong, Jian Pei, Jun Luo, and Haoyu Zhang. 2015. Finding Pareto Optimal Groups: Group-based Skyline. *Proceedings of the VLDB Endowment* 8, 13 (2015), 2086–2097. <https://www.vldb.org/pvldb/vol8/p2086-liu.pdf>
- [22] Peng Liu, Lemei Zhang, and Jon Atle Gulla. 2023. Pre-train, Prompt, and Recommendation: A Comprehensive Survey of Language Modeling Paradigm Adaptations in Recommender Systems. *Transactions of the Association for Computational Linguistics* 11 (2023), 1553–1571. https://doi.org/10.1162/tacl_a_00619
- [23] Bruce T. Lowerre. 1976. The HARP Speech Recognition System. In *Trends in Speech Recognition*. Prentice-Hall, 340–360.
- [24] Anh L. Mai, Pengyu Wang, Azza Abouzied, Matteo Brucato, Peter J. Haas, and Alexandra Meliou. 2024. Scaling Package Queries to a Billion Tuples via Hierarchical Partitioning and Customized Optimization. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1146–1158. <https://doi.org/10.14778/3641204.3641222>
- [25] Ernesto Queirós Vieira Martins. 1984. On a Multicriteria Shortest Path Problem. *European Journal of Operational Research* 16, 2 (1984), 236–245. [https://doi.org/10.1016/0377-2217\(84\)90077-8](https://doi.org/10.1016/0377-2217(84)90077-8)
- [26] Kaisa Miettinen. 1999. *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, Boston. <https://doi.org/10.1007/978-1-4615-5563-6>
- [27] Aditya G. Parameswaran, Petros Venetis, and Hector Garcia-Molina. 2011. Recommendation systems with complex constraints: A course recommendation perspective. *ACM Transactions on Information Systems* 29, 4 (2011), 20:1–20:33. <https://doi.org/10.1145/2037661.2037665>
- [28] Lu Qin, Jeffrey Xu Yu, and Lijun Chang. 2012. Diversifying Top-K Results. *Proceedings of the VLDB Endowment* 5, 11 (2012), 1124–1135. <https://doi.org/10.14778/2350229.2350233>
- [29] Sudeepa Roy, Sihem Amer-Yahia, Ashwin Chawla, Gautam Das, and Cong Yu. 2010. Constructing and Exploring Composite Items. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*. 843–854. <https://doi.org/10.1145/1807167.1807258>
- [30] Prabhakant Sinha and Andris A. Zoltners. 1979. The Multiple-Choice Knapsack Problem. *Operations Research* 27, 3 (1979), 503–515. <https://doi.org/10.1287/opre.27.3.503>
- [31] S. N. van Schaik, Judith Masthoff, and A. T. Wibowo. 2019. Package recommender systems: A systematic review. *Intelligent Decision Technologies* 13, 4 (2019), 435–452. <https://doi.org/10.3233/IDT-190140>
- [32] Min Xie, Laks V. S. Lakshmanan, and Peter T. Wood. 2012. Composite Recommendations: From Items to Packages. *Frontiers of Computer Science* 6, 3 (2012), 264–277. <https://doi.org/10.1007/s11704-012-2014-1>
- [33] Min Xie, Laks V. S. Lakshmanan, and Peter T. Wood. 2014. Generating Top-k Packages via Preference Elicitation. *Proceedings of the VLDB Endowment* 7, 14 (2014), 1941–1944.
- [34] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [35] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).
- [36] Tao Zhu, Patrick Harrington, Junjun Li, and Lei Tang. 2014. Bundle recommendation in ecommerce. In *Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 657–666. <https://doi.org/10.1145/2600428.2609603>
- [37] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haoan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2025. Large Language Models for Information Retrieval: A Survey. *ACM Transactions on Information Systems* 44, 1 (2025), 1–54. <https://doi.org/10.1145/3748304>

SUPPLEMENTARY MATERIAL

Our code and data are available at <https://github.com/yestincheng/groupbuy>.

A1 PROOF OF THEOREM 2.4 (EXPONENTIAL PARETO FRONTIER)

Theorem 2.4 (Exponential Pareto Frontier). *There are CGQ instances with n requirements whose Pareto frontier under $(Q, -\Delta C, R)$ contains 2^n objective vectors. Consequently, no algorithm can enumerate the frontier in time polynomial in the input size, independently of whether $P = NP$.*

PROOF. Fix n and let $M = \sum_{i=1}^n 2^i$. Construct a CGQ instance (G, B, α, β) with $G = \langle q_1, \dots, q_n \rangle$, any budget $B > 0$, any $\alpha \geq 0$, and $\beta = 0$. Requirement q_i has exactly two candidates. Both have price 0.

- x_i has $f_m(q_i, x_i) = 2^i/M$ and rating $r_{x_i} = 0$.
- y_i has $f_m(q_i, y_i) = 0$ and rating $r_{y_i} = 2^i$.

Both meet the threshold $\beta = 0$, so every one of the 2^n assignments is feasible. Each selects one item per requirement and has total cost $0 \leq (1 + \alpha)B$.

Let S_T denote the assignment selecting x_i for $i \in T$ and y_i otherwise, for $T \subseteq \{1, \dots, n\}$. Since all costs are zero, $C(S_T) = \Delta C(S_T) = 0$ for every T . By the definition of Q ,

$$Q(S_T) = \frac{1}{n} \sum_{i \in T} \frac{2^i}{M}, \quad R(S_T) = \sum_{i \notin T} 2^i,$$

so that $nM \cdot Q(S_T) + R(S_T) = \sum_{i=1}^n 2^i = M$ for every T .

Binary representations are unique, so $T \neq T'$ implies $\sum_{i \in T} 2^i \neq \sum_{i \in T'} 2^i$ and hence $Q(S_T) \neq Q(S_{T'})$. Suppose $S_{T'}$ dominated S_T under $(Q, -\Delta C, R)$. The cost components are equal, so domination would require $Q(S_{T'}) \geq Q(S_T)$ and $R(S_{T'}) \geq R(S_T)$ with at least one strict. But $nM \cdot Q + R$ is the same constant for both, so any strict increase in one component forces a strict decrease in the other, a contradiction. Hence all 2^n assignments are mutually nondominated and the Pareto frontier has size 2^n .

The instance contains $2n$ candidates. Across the candidates, the powers of two and rational matching values require $\Theta(n^2)$ bits in total. Any algorithm that outputs the frontier must write 2^n objective vectors. This output is exponential in the number of requirements and superpolynomial in the encoded input size. The conclusion is a statement about output size and does not depend on P versus NP . $\square$

Corollary 2.5 (Two objectives suffice). *There are CGQ instances with n requirements whose Pareto frontier under $(Q, -\Delta C)$ alone contains 2^n objective vectors with pairwise distinct cost overheads. On these instances, any method that returns one assignment per ILP run, including the ϵ constraint method, needs at least 2^n runs to return the complete frontier.*

PROOF. Fix n and let $M = \sum_{i=1}^n 2^i$. Construct a CGQ instance with $G = \langle q_1, \dots, q_n \rangle$, target budget $B = 1$, budget-relaxation parameter $\alpha = M$, and $\beta = 0$. Requirement q_i has exactly two candidates. Candidate x_i has $f_m(q_i, x_i) = 2^i/M$ and price 2^i , and

candidate y_i has $f_m(q_i, y_i) = 0$ and price 0. All ratings are 0. For $T \subseteq \{1, \dots, n\}$, let S_T select x_i for $i \in T$ and y_i otherwise. Its total cost is $C(S_T) = \sum_{i \in T} 2^i \leq M \leq (1 + \alpha)B$, so every one of the 2^n assignments is feasible. Its solution quality is $Q(S_T) = \sum_{i \in T} 2^i / (nM)$, and its cost overhead is $\Delta C(S_T) = \max(C(S_T) - 1, 0)$, which equals $C(S_T) - 1$ for nonempty T because $C(S_T) \geq 2$, and 0 for $T = \emptyset$.

Binary representations are unique, so distinct subsets have distinct total costs, hence distinct solution quality values and distinct cost overheads. Suppose $S_{T'}$ dominated S_T . If both subsets are nonempty, then $nM Q(S_{T'}) - \Delta C(S_{T'}) = nM Q(S_T) - \Delta C(S_T) = 1$, so a strict improvement in one objective forces a strict loss in the other, a contradiction. If $T = \emptyset$, then S_T has the unique minimum cost overhead 0, so no assignment dominates it. If $T' = \emptyset$, then $Q(S_{T'}) = 0 < Q(S_T)$, so $S_{T'}$ dominates nothing. Hence all 2^n assignments are mutually nondominated under $(Q, -\Delta C)$, and adding the constant rating objective does not change the frontier.

Each ILP run of the ϵ constraint method returns at most one objective vector, so at least 2^n runs are needed. In the two objective case, the run with cost overhead threshold ϵ_C returns the feasible assignment with the largest solution quality among those with $\Delta C \leq \epsilon_C$, so distinct frontier points additionally require distinct thresholds. In particular, the grid of g cost overhead thresholds over $[0, \alpha B]$ in Section 3.1.2 needs $g \geq 2^n$ on this instance. $\square$

A2 PROOF OF THEOREM 2.7 (COMPLEXITY OF WEIGHTED CGQ)

Theorem 2.7 (Complexity of Weighted CGQ). *Given a threshold τ , deciding whether there exists a feasible assignment O_G such that $F(O_G) \geq \tau$ is NP-complete. Consequently, finding an optimal assignment for Weighted CGQ is NP-hard.*

PROOF. We assume that all numeric inputs are rational values encoded in binary. Membership in NP follows because we can verify in polynomial time that an assignment selects one candidate for every requirement, satisfies the matching and budget constraints, and has objective value at least τ .

For NP hardness, we reduce from the decision version of the Multiple-Choice Knapsack Problem [30]. An instance contains n groups of items. Each item j has a nonnegative weight w_j and profit p_j . The question is whether one item can be selected from every group so that their total weight is at most a capacity W and their total profit is at least a target P . We consider instances in which at least one item has positive profit, so the maximum profit is positive.

We create one requirement q_i for each group and one candidate o_j for each item in that group. We set $c_{o_j} = w_j$, $r_{o_j} = p_j$, and $f_m(q_i, o_j) = 1$. We also set $B = W$, $\alpha = 0$, $\beta = 0$, $\lambda_1 = \lambda_2 = 0$, and $\lambda_3 = 1$. Because $\alpha = 0$, the cost overhead term is omitted. The resulting objective is $F(O_G) = R(O_G)/R_{\max}$. We set the decision threshold to $\tau = P/R_{\max}$.

Every feasible CGQ assignment selects exactly one candidate from each requirement, so it corresponds to selecting one item from each knapsack group. The CGQ budget constraint is satisfied exactly when the knapsack capacity constraint is satisfied. Moreover, $F(O_G) \geq \tau$ exactly when the selected items have total profit at least P . The construction is polynomial, and the two decision instances

therefore have the same answer. Together with membership in NP, this proves NP completeness. $\square$

Proposition 2.8 (Pseudopolynomial Weighted CGQ). *Suppose all numeric inputs are rational values encoded in binary, and all prices are nonnegative integers. Let $K = \lfloor (1 + \alpha)B \rfloor$. Weighted CGQ can be solved exactly using $O((K + 1) \sum_{i=1}^n |C_i|)$ arithmetic operations and comparisons, while storing $O(n(K + 1))$ table entries. The algorithm is pseudopolynomial in K , so the integer price formulation of Weighted CGQ is NP-hard only in the weak sense.*

PROOF. Let $K = \lfloor (1 + \alpha)B \rfloor$. For an assignment O_G , write $A(O_G) = \lambda_1 \sum_i f_m(q_i, o_{q_i})/n + \lambda_3 \sum_i r_{o_{q_i}}/R_{\max}$ for the additive part of the weighted objective, so that $F(O_G) = A(O_G) - \lambda_2 \max(C(O_G) - B, 0)/(\alpha B)$ depends on O_G only through $A(O_G)$ and $C(O_G)$. For $i \in \{0, \dots, n\}$ and $c \in \{0, \dots, K\}$, let $T[i][c]$ be the maximum of the additive part over partial assignments of $q_1, \dots, q_i$ with total cost exactly c , with $T[i][c] = -\infty$ when no such partial assignment exists and $T[0][0] = 0$. Because prices are nonnegative integers and the additive part decomposes over requirements,

$$T[i][c] = \max_{o \in C_i, c_o \leq c} \left(T[i-1][c - c_o] + \frac{\lambda_1 f_m(q_i, o)}{n} + \frac{\lambda_3 r_o}{R_{\max}} \right).$$

Every feasible assignment has an integer total cost in $\{0, \dots, K\}$, and every partial assignment counted in $T[i][c]$ extends to a feasible assignment only if its cost stays within K , so the optimal value of Weighted CGQ is $\max_{c \leq K} (T[n][c] - \lambda_2 \max(c - B, 0)/(\alpha B))$, with the overhead term omitted when $\alpha = 0$. An optimal assignment follows from the argument of each maximum. Filling row i examines every candidate in C_i for each of the $K + 1$ cost values, which gives the stated bound on arithmetic operations and comparisons. The table stores $(n + 1)(K + 1)$ entries, each containing an objective value and a pointer for reconstructing an optimal assignment.

For an input of bit length L , every finite table value is a sum of at most n rational candidate contributions. These sums and the values used to evaluate the cost overhead term have bit lengths polynomial in L . Exact arithmetic and comparisons therefore cost a factor polynomial in L , and each table entry requires polynomially many bits in L . The bit complexity is polynomial in L and K . In particular, the algorithm runs in polynomial time when K is polynomially bounded in L .

The same dynamic program handles rational prices after scaling them to integers. Choose a common denominator D of the prices and replace each price c_o by Dc_o and the target budget B by DB . This scaling preserves the feasible assignments and the weighted objective because both cost overhead and its normalizing factor αB scale by D , while solution quality and total rating are unchanged. The dynamic program then uses the integer capacity $K_D = \lfloor D(1 + \alpha)B \rfloor$. A common denominator can be computed with bit length polynomial in L , but its numeric value, and hence K_D , can be large even when the original prices are small. The bounds for rational prices therefore depend on K_D , and polynomial running time follows when K_D is polynomially bounded in L . $\square$

The cost index of the table also bounds the Pareto CGQ output. Two feasible assignments with equal total cost have equal cost overhead, so the frontier under $(Q, -\Delta C, R)$ contains at most $\sum_{c=0}^K |\text{SKY}_c|$ vectors, where SKY_c is the set of nondominated (Q, R) pairs among

feasible assignments with total cost exactly c . This bound does not contradict Theorem 2.4, whose instance places all 2^n frontier points in the single cost cell $c = 0$, nor Corollary 2.5, whose instance uses 2^n distinct cost cells.

A3 COMPLEXITY AND CANDIDATE REDUCTION

Let $n = |G|$ denote the number of slots.

PBS. Let $m_i = |C_i|$ denote the size of the reduced candidate set for requirement q_i , and let $m = \max_i m_i$. At search level i , PBS maintains at most b partial assignments, each generating at most m_i feasible extensions. Let $k_i \leq b m_i$ be the number of extensions generated at that level. Computing the three comparison values costs $O(k_i)$ when each value is updated incrementally from its parent. Pairwise dominance filtering would cost $O(k_i^2)$. We instead use three dimensional maxima filtering [20], implemented by sorting and a bounded scan of retained entries. If the scan exceeds $k_i \lceil \log_2(k_i + 1) \rceil$ comparisons, a Fenwick tree computes the maxima over the full input. For finite comparison values, filtering therefore costs $O(k_i \log(k_i + 1))$ time and $O(k_i)$ auxiliary space. After dominance filtering, the implementation sorts the remaining partial assignments for beam truncation in $O(k_i \log(k_i + 1))$ time. Copying the selected item identifiers into each extension costs $O(ik_i)$ at depth i , even though its comparison values are updated incrementally.

Let N be the total number of candidates before reduction. Applying the same maxima filter separately to each retrieved candidate set costs at most $O(N \log(N + 1))$. Testing extensions costs at most $O(\sum_i b m_i)$, and $k_i \leq b m_i$. Including filtering, sorting, and assignment copying, the total time after retrieval is therefore

$$O(N \log(N + 1) + nbm (\log(bm + 1) + n)).$$

The implementation computes the final frontier from the complete beam with one maxima filter in $O(b \log(b + 1))$ time and maps the selected item identifiers to the retrieved candidate sets in $O(nb)$ time. These costs are absorbed by the bound.

The retained beam contains at most b partial assignments, while a materialized expansion level contains at most $k_i \leq b m_i$ partial assignments before filtering. Working memory is therefore $O(bm)$ partial assignment records in the uniform case, excluding the candidate lists. If every record explicitly stores up to n selected item identifiers, the corresponding bound is $O(bmn)$. For PBS without beam truncation, these bounds use the actual beam size at each level in place of b and may become exponential.

PTS. Let M denote the iteration budget and $m = \max_i |C_i|$. Let $|P|$ be the largest frontier size and $D = \max_d |D[d]|$. D is listed separately because it counts mutually nondominated *partial assignments* at one depth, which is not bounded by $|P|$ and is potentially larger. Each iteration performs selection, expansion, greedy completion, frontier update, and exhaustion propagation. The selection path has length at most n . DFS scans stored child lists during selection. Selection and exhaustion propagation therefore cost $O(nm)$ in the worst case. Initializing and sorting a node's candidate list costs $O(m \log(m + 1))$ once per created node. Candidates are removed from the front of this list in $O(1)$ time using a

deque. Copying a child's partial assignment costs $O(n)$. The greedy completion scans at most m candidates for each of at most n remaining requirements and costs $O(nm)$. At the selected expansion node, one iteration may discard as many as m candidates using the objective upper bound and partial assignment dominance. These comparisons cost $O(m(|P|+D))$, and a frontier update costs $O(|P|)$ in the worst case. A single iteration therefore costs $O(nm + m \log(m+1) + m(|P|+D) + |P| + n)$ and M iterations cost

$$O(M(nm + m \log(m+1) + m(|P|+D) + |P| + n)),$$

since at most $M+1$ nodes are created and each candidate list is initialized once. This excludes retrieval and candidate reduction. Initial ordering of the reduced candidates costs $O(nm \log(m+1))$, and suffix bounds cost $O(nm)$. Final maxima filtering costs $O(|P| \log(|P|+1))$, and mapping the returned assignments to original candidate indices costs $O(n|P|)$. The frontier and partial assignment archives are stored by the cost coordinate, so a dominance scan visits only entries that satisfy the cost condition. This ordering reduces comparisons in practice without changing the worst case.

The search tree contains $O(M)$ nodes. Node storage is $O(Mn)$ because the reference implementation keeps the accumulated partial assignment at each node. Including the candidate sets, partial assignment archives, frontier, and memoized completion suffixes, total space is $O(M(n+m) + nm + nD + |P|)$, excluding the original retrieved candidate lists. Each node stores up to m untried candidate indices, contributing $O(Mm)$ space. At most one completion suffix and one complete assignment are produced per iteration, so storing their choices costs $O(Mn)$. The archive stores references to created nodes, using $O(nD)$ space. With the optional candidate cap, the search terms use $\min\{m, \kappa\}$, while candidate reduction, initial ordering, and storage of the full reduced candidate sets retain their dependence on uncapped sizes.

Candidate reduction. Within one requirement, suppose item o' has at least the matching quality and rating of item o and costs no more. Replacing o by o' in any assignment preserves each matching-quality constraint and the relaxed budget cap while weakly improving all three Pareto objectives $(Q, -\Delta C, R)$. Thus o cannot be necessary for a distinct Pareto optimal objective vector, which establishes the soundness of the three dimensional reduction used by PBS and PTS. Two details make this argument complete. First, the reduction removes only candidates that are strictly dominated in the componentwise order on $(f_m, -c, r)$, and among candidates with identical triples it retains exactly one. Since strict dominance is a strict partial order on a finite set, every removed candidate is weakly dominated by a retained candidate, either strictly or as an identical duplicate, and the retained set is the set of maxima of the triples. Second, given any Pareto optimal assignment over the original candidate sets, replacing every removed item by a retained item that dominates it yields a feasible assignment over the reduced sets whose objective vector weakly dominates the original vector and therefore equals it. Hence every Pareto optimal objective vector over the original candidate sets is attained over the reduced candidate sets, and the two frontiers coincide.

A4 PROOF OF LEMMA 3.1

Lemma 3.1 (Partial Assignment Dominance). *Let v and v' be partial assignments at the same depth with the same remaining candidate sets. If $Q_{v'} \geq Q_v$, $C_{v'} \leq C_v$, and $R_{v'} \geq R_v$, then every feasible completion of v has a feasible completion from v' with an objective vector that is at least as good under $(Q, -\Delta C, R)$.*

PROOF. Consider any feasible suffix σ of v . The partial assignments have the same remaining candidates, so σ is also available to v' . Since $C_{v'} \leq C_v$, appending σ to v' cannot violate the relaxed budget cap when appending it to v is feasible. The same suffix adds equal contributions to both partial assignments. Thus $v'\sigma$ has at least the quality and rating of $v\sigma$ and no greater raw accumulated cost. Because $\Delta C = \max(C - B, 0)$ is monotone in C , $v'\sigma$ is also at least as good under $(Q, -\Delta C, R)$. If the final objective vectors are equal, retaining $v'\sigma$ still preserves a representative of that vector. $\square$

A5 PROOF OF THEOREM 3.2 (PARETO COMPLETENESS OF PBS)

Theorem 3.2 (Pareto Completeness of PBS). *If PBS performs no beam truncation and prunes partial assignments by $(Q, -C, R)$, it returns a representative of every Pareto optimal objective vector over the supplied candidate sets.*

PROOF. Let z be a Pareto optimal objective vector over the reduced candidate sets and let $O^* = \langle o_1^*, \dots, o_n^* \rangle$ be a feasible assignment with objective vector z . Call a partial assignment v of $q_1, \dots, q_t$ useful for z if some feasible completion of v has an objective vector that weakly dominates z . We prove by induction on t that the beam $\mathcal{B}_t$ after level t contains a partial assignment useful for z . The empty assignment is useful because O^* completes it, so the claim holds for $t = 0$.

Assume that $\mathcal{B}_{t-1}$ contains a partial assignment v useful for z , and let $v\sigma$ be a feasible completion of v whose vector weakly dominates z . Let u be v extended with the first item of σ . Prices are nonnegative, so $C_u \leq C(v\sigma) \leq (1 + \alpha)B$, and u is among the feasible extensions generated at level t . The remaining suffix of σ completes u to $v\sigma$, so u is useful for z . If u survives dominance filtering, the claim holds for t . Otherwise u is strictly dominated under $(Q, -C, R)$ by another extension. Strict dominance is a strict partial order on the finite set of extensions, so following dominating elements reaches a nondominated extension u' that dominates u by transitivity, and u' is retained by the filter. Lemma 3.1 applied to u and u' yields a feasible completion of u' whose vector weakly dominates that of $v\sigma$ and hence z . Thus u' is useful for z . Since PBS performs no beam truncation, every retained extension enters $\mathcal{B}_t$, which proves the claim for t .

After level n , the beam contains a complete feasible assignment whose objective vector weakly dominates z . Because z is Pareto optimal over the same candidate sets, this vector equals z . The final filter under $(Q, -\Delta C, R)$ removes only dominated vectors and keeps one representative per vector, so it returns a representative of z . Applying the argument to every Pareto optimal vector proves the

Algorithm 3 PTS selection and exhaustion helpers

```

1: function SELECTUNEXHAUSTED( $r$ )
2:    $v \leftarrow r$ 
3:   while  $d_v < n$  and  $v$  is not exhausted do
4:     if  $A(v) = \perp$  then
5:       Initialize  $A(v)$  using reachability and  $F'$  ordering
6:     end if
7:     if  $v$  has no unexhausted child then
8:       return  $v$ 
9:     end if
10:     $v \leftarrow$  first unexhausted child of  $v$ 
11:  end while
12:  return  $v$ 
13: end function
14: function PROPAGATEEXHAUSTION( $v$ )
15:   while  $v \neq \text{null}$  do
16:     if  $v$  is not exhausted and does not meet the exhaustion condition then
17:       return
18:     end if
19:     Mark  $v$  exhausted
20:      $v \leftarrow \text{parent}(v)$ 
21:   end while
22: end function

```

Algorithm 4 PTS child creation and archive insertion helpers

```

1: function NEXTUNPRUNEDEXTENSION( $v, P, D, F'$ )
2:   while  $A(v) \neq \emptyset$  do
3:     Remove the first candidate from  $A(v)$ 
4:     Form its extension  $s$  of  $v$  at depth  $d = d_v + 1$ 
5:     if  $d < n$  and some  $p \in P$  weakly dominates  $\text{ub}(s)$  then
6:       continue
7:     end if
8:     if some entry in  $D[d]$  weakly dominates  $s$  under  $(Q, -C, R)$  then
9:       continue ▷ do not modify  $D[d]$ 
10:    end if
11:    Create  $u$  for  $s$  with parent  $v$  and append it to  $v$ 's children
12:    Remove entries in  $D[d]$  weakly dominated by  $u$  under  $(Q, -C, R)$ 
13:    Insert a reference to  $u$  into  $D[d]$  in cost order
14:    return  $u$ 
15:  end while
16:  return null
17: end function

```

theorem over the reduced candidate sets, and the candidate reduction argument in Section A3 extends it to the supplied candidate sets. $\square$

A6 PRUNING AND PARETO COMPLETENESS OF PTS

For a node v , let $A(v)$ be its ordered list of untried candidates, with $A(v) = \perp$ indicating that the list has not been initialized. Initialization retains extensions satisfying Equation (2) and orders them by decreasing F' . Children are stored in creation order. New nodes are unexhausted and unevaluated, with no children and $A(v) = \perp$. Algorithm 3 selects a node and propagates exhaustion. Algorithm 4 creates at most one child and updates the archive. These helpers use the same fixed candidate sets and suffix bounds throughout the search. The exhaustion condition holds when a terminal node has been evaluated, or when an internal node has an initialized empty candidate list and all its children are exhausted. Removing an archive entry leaves its tree node unchanged.

Theorem 3.3 (Soundness of PTS Pruning). *Let v be a partial assignment in the fixed reduced search space. Pruning v preserves a representative of every Pareto optimal objective vector if Equation (2)*

fails, if an objective vector in P weakly dominates $\text{ub}(v)$, or if a partial assignment in $D[d_v]$ weakly dominates v under $(Q, -C, R)$.

PROOF. If Equation (2) fails, even the minimum cost suffix exceeds the relaxed budget cap. The remaining requirements are independent and every retained candidate satisfies the matching threshold, so v has no feasible completion.

For the second condition, every feasible completion of v has quality at most $Q_v + \max Q[d_v + 1]$, cost at least $C_v + \min C[d_v + 1]$, and rating at most $R_v + \max R[d_v + 1]$. Its objective vector is therefore weakly dominated by $\text{ub}(v)$. An objective vector in P that weakly dominates this upper bound also weakly dominates every completion of v . Any equal objective vector already has a representative assignment. The frontier changes later, but `UpdateFrontier` removes a vector only when it inserts a vector that dominates it. By induction over the updates, every vector that was ever in P is weakly dominated by some vector in the frontier at every later time. A Pareto optimal vector z reachable below v satisfies $z \leq \text{ub}(v) \leq p$ for the vector $p \in P$ used at pruning time, and the final frontier contains a feasible vector $p' \geq p \geq z$, which equals z by Pareto optimality.

For the third condition, fixed candidate sets give both partial assignments the same suffix choices. Lemma 3.1 maps every feasible completion of v to a completion of its archive representative that is at least as good under $(Q, -\Delta C, R)$. By the archive invariants, that representative v' is a created tree node, and if the archive later replaces v' , the replacement is a created tree node that weakly dominates v' and hence v under $(Q, -C, R)$. Any Pareto optimal vector z attained by a completion of v is therefore attained by a completion of a created node at the same depth, which remains in the tree and is the representative that the archive keeps. In all three cases, every Pareto optimal vector attained below v is either already weakly dominated by a vector in P or attained below a created node at depth d_v . $\square$

Theorem 3.4 (Pareto Completeness of PTS). *Let $\mathcal{T}$ be the feasible assignments formed from the fixed reduced candidate sets, let $|V| = \sum_{d=0}^n \prod_{i \leq d} |C_i|$ be the number of nodes of the fixed reduced tree. Assume exact arithmetic. At every iteration, P contains the non-dominated objective vectors of the complete assignments evaluated so far. Continued search exhausts the root after at most $2|V|$ iterations. At root exhaustion, P is the complete frontier over $\mathcal{T}$. Candidate reduction preserves this frontier over the original candidate sets.*

PROOF. For an assignment set X , let $\text{vec}(X)$ be its objective vectors and let $\text{ND}(\text{vec}(X))$ be their nondominated subset.

Invariant. Let $\mathcal{T}_M \subseteq \mathcal{T}$ be the assignments evaluated after M iterations, including assignments obtained by greedy completion, and let P_M be the maintained frontier. Each update inserts the new objective vector and removes the vectors it dominates. Thus, $P_M = \text{ND}(\text{vec}(\mathcal{T}_M))$, which proves the first statement.

Termination. The fixed candidate sets define a finite tree with $|V|$ nodes. A terminal node is exhausted after evaluation. An internal node is exhausted only after its candidate list has been initialized, all its candidates have been expanded or pruned, and all created children are exhausted. Selection initializes a node's candidate list when first needed and descends only into unexhausted children.

DFS follows the first unexhausted child and returns the current node when no such child remains. A node whose list is already fully processed is either exhausted or has an unexhausted child through which selection continues. Since descent increases depth in a finite tree, selection returns a node with an untried candidate or a newly initialized empty candidate list.

Consider an iteration selecting node v . Either `NextUnprunedExtension` creates a new child, or it finishes processing the remaining candidates without creating a child. The second case includes a list that is empty when initialized. Creating a child occurs at most $|V| - 1$ times in total, once per nonroot node. Finishing a candidate list without creating a child occurs at most once per node, since the list is initialized only once and subsequent selection either skips the exhausted node or descends through an unexhausted child. Exhaustion is propagated immediately after terminal evaluation or when an internal node's initialized candidate list is empty and all its children are exhausted. Consequently, continued search exhausts the root after at most $2|V|$ iterations.

Soundness of exhaustion. A terminal node is exhausted only after it is evaluated. A branch pruned by the reachability bound, objective upper bound, or partial assignment dominance satisfies Theorem 3.3. An internal node is exhausted only after all of its candidates have been expanded or pruned and all created children are exhausted. Induction over its children therefore ensures that every completion below the node has been evaluated or pruned by a sound rule.

Frontier completeness at exhaustion. Let P denote the frontier at root exhaustion and let z be a Pareto optimal vector over $\mathcal{T}$. Call a node *closed* if it is an exhausted created node or a candidate extension rejected by a pruning rule. At root exhaustion every created node is exhausted, and every candidate extension of a created node is a created node or a pruned extension, so every node of the fixed tree that is a child of a created node is closed. We prove by downward induction on the depth d , from n to 0, that if z is attained by a completion of a closed node v at depth d , then $z \in P$.

At depth n , a closed node is either an evaluated terminal node, in which case its vector was evaluated and by the invariant a vector weakly dominating it remains in P , which equals z by Pareto optimality, or a pruned terminal extension, which is handled below. Let v be closed at depth $d < n$. If v is exhausted, the completion attaining z passes through a candidate extension u of v at depth $d + 1$, and u is closed, so the induction hypothesis gives $z \in P$. If v was pruned by the reachability bound, it has no feasible completion, and the case is vacuous. If v was pruned by the objective upper bound against a vector p that was in P at that time, then $z \leq \text{ub}(v) \leq p$, and by the invariant the final P contains a feasible vector $p' \geq p \geq z$, so $p' = z \in P$. If v was pruned by partial assignment dominance, the archive invariants give a created node v' at depth d that weakly dominates v under $(Q, -C, R)$, where v' may have the same comparison values as v , and Lemma 3.1 gives a completion of v' whose vector weakly dominates z and hence equals z . Since v' is created, it is exhausted at root exhaustion, and the argument for exhausted nodes at depth d applies to v' , which uses only closed nodes at depth $d + 1$. Pruned terminal extensions at depth n are covered by the same three pruning cases, where the exhausted node case cannot occur. This completes the induction.

The root is exhausted and every assignment in $\mathcal{T}$ is a completion of it, so every Pareto optimal vector over $\mathcal{T}$ is in P . Conversely, suppose some $p \in P$ were dominated by the vector of an assignment in $\mathcal{T}$. That vector is weakly dominated by a Pareto optimal vector z over $\mathcal{T}$, which lies in $P \subseteq \text{vec}(\mathcal{T}_M)$ and dominates p , contradicting $P = \text{ND}(\text{vec}(\mathcal{T}_M))$. Hence $P = \text{ND}(\text{vec}(\mathcal{T}))$. Candidate reduction preserves this frontier over the original candidate sets by the argument in Section A3. $\square$

For uncapped PTS, root exhaustion establishes that the full reduced search space has been processed. When the optional candidate cap is used, this conclusion applies only to the capped search space.

For the optional candidate cap, we order candidates $o \in C_i$ by decreasing $\lambda_1 f_m(q_i, o)/n + \lambda_3 r_o/R_{\max}$, using the guidance weights of F' and retaining their input order for ties. This ordering is independent of the partial assignment, so every node at the same depth uses the same capped candidate set. The shared candidate sets preserve the available suffix assignments required by the partial assignment dominance argument. Within these sets, expansion still orders candidates by the F' values of the resulting partial assignments.

The implementation used for the experiments also accumulates completion proxy values and visit counts along the ancestor path. These $O(n)$ updates do not affect depth first selection.

The pruning and completeness arguments assume exact arithmetic. The implementation uses floating point arithmetic with an absolute feasibility tolerance of 10^{-9} . It accumulates objective values in requirement order and uses unrounded values in the partial assignment archives and maintained frontier. Final dominance filtering and duplicate removal compare objective values rounded to nine decimal places. Rollout memoization is keyed by depth, the stored floating point cost without rounding, and the scalarization direction. A cached completion is reconstructed in requirement order and checked against the relaxed budget cap with the feasibility tolerance before evaluation. Root exhaustion indicates that the tree has been processed under these comparisons, which may differ from exact arithmetic at numerical boundaries.

A7 WEIGHTED CGQ ILP FORMULATION

When the weights in Equation (1) are known, the output of Weighted CGQ is one assignment, so there is no need to approximate a frontier. The multiple choice structure maps directly to an integer linear program. We create one binary variable for each eligible item, require exactly one selected item per requirement, and impose the relaxed budget cap on their total price. The only nonlinear term is the cost overhead $\Delta C = \max(C - B, 0)$. One nonnegative auxiliary variable linearizes it.

Formally, let C_i contain the items for requirement q_i that pass the matching threshold, let binary $x_{i,o}$ indicate that item $o \in C_i$ is selected, and let $y \geq 0$ represent cost overhead. The resulting ILP is

$$\begin{aligned} \max_{x,y} \quad & \frac{\lambda_1}{n} \sum_i \sum_{o \in C_i} f_m(q_i, o) x_{i,o} \\ & + \frac{\lambda_3}{R_{\max}} \sum_i \sum_{o \in C_i} r_o x_{i,o} - \frac{\lambda_2}{\alpha B} y, \end{aligned} \quad (\text{ILP})$$

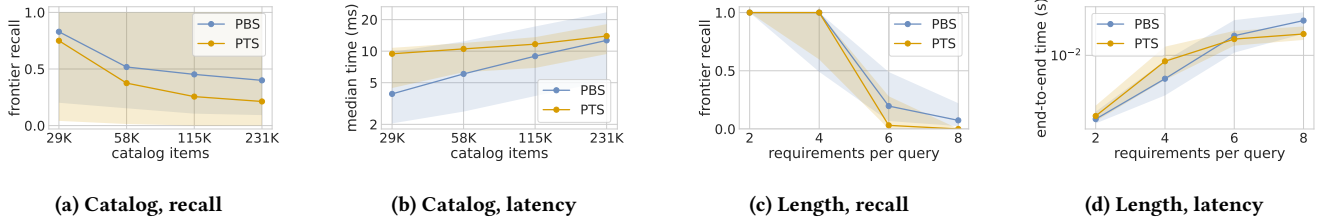


Figure A1: Catalog size and query length on Airbnb. Medians with interquartile ranges.

$$\begin{aligned}
\text{s.t. } & \sum_{o \in C_i} x_{i,o} = 1 & \forall i, \\
& \sum_i \sum_{o \in C_i} c_o x_{i,o} \leq (1 + \alpha)B, \\
& y \geq \sum_i \sum_{o \in C_i} c_o x_{i,o} - B, \\
& x_{i,o} \in \{0, 1\}, \quad y \geq 0.
\end{aligned}$$

For $\alpha = 0$ the overhead is fixed to zero and its objective term is omitted. When $\lambda_2 > 0$, the negative coefficient makes y equal $\max(C - B, 0)$ at an optimum. When $\lambda_2 = 0$, the value of y does not affect the objective. In both cases, the selected assignment optimizes Definition 2.6. The formulation provides the exact baseline for Weighted CGQ. Its multiple choice structure also supports a compact specialized branch-and-bound implementation, which we verify against HiGHS and exhaustive enumeration.

Replacing Pareto retention by ranking on F gives a weighted specialization of the search methods. On our measured workloads, the exact ILP baseline returns an optimum with lower median end-to-end time than these specializations. One ILP run returns one assignment for one supplied preference vector. It does not answer the interaction in which an agent must present alternatives before those weights are known.

A8 WORKLOAD GENERATION

We generate query pools by multiplying the sum of minimum eligible item prices for the requirements by a factor sampled uniformly from $[1.1, 1.5]$ with probability 0.6 and from $[0.8, 0.99]$ otherwise. We round the result to an integer target budget.

For Amazon, we generate natural language requirements from product keywords and combine them across product categories. Budget generation uses minimum eligible prices within each requirement’s source category. From a pool of 1,000 queries, we select 100 with nonempty retrieved candidate sets and $C_{\min} \leq B$, ensuring feasibility at $\alpha = 0$. We retain their original target budgets. At $\beta = 0.8$, the minimum cost used for selection is $C_{\min} = \sum_i \min_{o \in C_i} c_o$, computed over the retrieved candidate sets, which may include items from other categories. The budget sampling probabilities apply to the generated pool before selection.

For Airbnb, we generate templated stay requirements from listing attributes and combine requirements for distinct cities. Each assignment selects one listing per city. For budget generation, each requirement’s minimum eligible price is rounded to a positive integer before summation, and the resulting target budget is also

rounded to a positive integer. At $\beta = 0.8$, 99.8% of the retrieved matches belong to the required city.

A9 ADDITIONAL EXPERIMENTAL RESULTS

A9.1 Search Budgets

Figure A2 reports the end to end latency of PBS and PTS on Amazon as the beam width b and the iteration budget M vary. The median latency of PBS stays between 1.1 and 3.0 ms for every beam width, while its upper quartile rises from 1.4 ms at $b = 1$ to 6.7 ms at $b = 500$. The median latency of PTS is 4.1 ms at $M = 200$ and stays between 6.5 and 6.7 ms from $M = 500$ to $M = 8000$, while its upper quartile rises from 10.1 ms at $M = 500$ to about 16.5 ms at larger budgets. Larger budgets therefore mainly increase the latency of the slower queries.

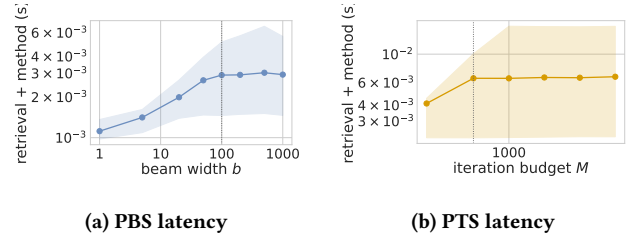


Figure A2: Latency of PBS and PTS as b and M vary on Amazon. Medians with interquartile ranges.

A9.2 Additional Scalability Study

A9.2.1 Catalog Size and Query Length on Airbnb. We vary Airbnb catalog size and query length to assess their effect on PBS and PTS frontier recall and latency, as shown in Figure A1.

We use nested catalogs with 29K to 231K listings. Median recall decreases from 0.83 to 0.40 for PBS and from 0.75 to 0.21 for PTS, while median latency increases from 3.9 to 12.7 ms for PBS and 9.4 to 13.9 ms for PTS. With fixed search budgets, the methods examine a smaller fraction of the candidate combinations as the catalog grows. At the largest catalogs, PBS has higher recall than PTS on Airbnb, while PTS has higher recall on Amazon.

For query length, median recall is 1.00 for both methods at lengths 2 and 4, then decreases to 0.074 for PBS and 0.000 for PTS at length 8. Each additional requirement adds a search level and increases the number of candidate combinations, so the fixed beam width and iteration budget cover a smaller fraction of the search space.

A9.2.2 Catalog Latency with Complete Frontier Search. We compare PBS, PTS, PBS without beam truncation, and DP as catalog size increases on Amazon and Airbnb in Figure A3. Each method’s latency includes retrieval, which is plotted separately.

On Amazon at 2.14M items, PBS without beam truncation recovers the complete frontier at a median of 20.7 ms, close to PBS at 19.8 ms, while DP takes 70.6 ms. On Airbnb at 231K listings, DP takes 84.2 ms compared with 14.7 ms for PBS without beam truncation. Both complete frontier methods finish every query at each catalog size. PBS without beam truncation thus retains median latency close to bounded PBS on these workloads, while DP has higher median latency.

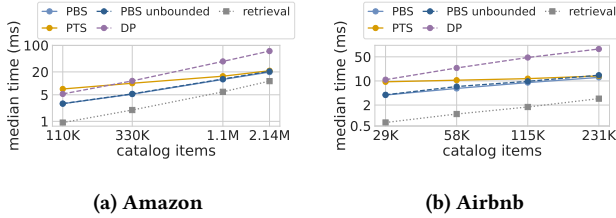


Figure A3: Median latency as catalog size increases on Amazon and Airbnb. Retrieval is plotted separately.

A9.2.3 Retrieval and Search Time by Catalog Size. We separate retrieval and search time within total latency for PBS and PTS across catalog sizes, as shown in Figure A4. The measurements use the same runs as the catalog size figures, with 100 queries per setting. The nominal 330K Amazon catalog contains 298,233 items. Search time includes search initialization. Total latency also includes candidate reduction, final frontier filtering, assignment reconstruction, and method invocation overhead. Each curve reports medians, which need not sum to the median total latency.

Query embeddings and resident catalog embeddings are prepared before timing. Retrieval includes query transfer to the GPU, similarity and price filtering, and candidate materialization on the CPU. The GPU uses a float16 prefilter at threshold $\beta - 0.01$ and float32 rescoring against the matching threshold. The device is synchronized before the timer starts and before it stops. Total latency is the sum of retrieval and method invocation time for each query. Reference construction for another method, metric calculation, and writing results to files are excluded. The ϵ -ILP runs use SciPy 1.17.1 with HiGHS 1.12.0 configured for one solver thread. The Weighted CGQ ILP runs use the specialized Python branch and bound solver.

At 2.14M Amazon items, median retrieval takes 11.2 ms for PBS and 11.4 ms for PTS, exceeding their search times of 4.6 and 7.9 ms. At 231K Airbnb listings, retrieval takes 3.6 and 3.1 ms, while PBS and PTS search take 5.3 and 9.0 ms, respectively. Retrieval scans the full catalog, while search processes the reduced candidate sets. Retrieval therefore accounts for more time on the largest Amazon catalog, whereas search accounts for more on Airbnb.

A9.2.4 Memory by Catalog Size. We compare the sampled increase in host memory during search across four Amazon catalogs, as shown in Table A1. Each method runs in a separate process holding the candidate inputs for 100 queries. The ϵ -ILP runs use $g = 4$ and

$g = 16$ with a 60 second query limit. At $g = 16$, 7 queries reach that limit at 1.1M items and 19 at 2.14M items. With fixed 110K budgets, DP reaches the 60 second limit on 3 of 100 queries at 2.14M Amazon items. It reaches the limit on 1 of 100 queries in the default Airbnb workload.

At 2.14M Amazon items, host RSS after retrieval is 28.8 GB, including resident catalog structures and embedding matrices, and peak GPU allocation is 16.3 GB. At 231K Airbnb listings, the corresponding values are 3.9 GB and 1.8 GB.

The sampled search increase for PBS stays below 1 MB at every Amazon scale. At 2.14M items, it is 0.81 MB for PBS, compared with 1.00 MB for DP, 1.99 MB for PTS, 7.94 MB for PBS without beam truncation, and 43.91 MB for ϵ -ILP with $g = 16$. PBS and DP have similar sampled increases, while unbounded PBS uses more. Differences in state representation prevent attributing this gap solely to beam truncation.

A9.2.5 Candidate Cap and Budget Protocol. We measure whether the default candidate cap limits PTS frontier recall. On the 100 default queries per dataset with $M = 500$, $\kappa = 50$ and uncapped PTS both attain median recall 1.000 on Amazon and 0.093 on Airbnb. On Amazon, all returned frontiers are identical, and median candidate evaluations remain 1,795.

We also vary the PTS candidate cap at 2.14M Amazon items, as shown in Figure A5. $\kappa = 50$ gives mean recall 0.523, compared with 0.524 for uncapped PTS, whereas $\kappa = 25$ decreases to 0.50 and $\kappa = 10$ to 0.25. The default cap accounts for little of the frontier recall loss at the largest scale. Most of the loss persists without the cap under the fixed iteration budget.

We compare target budgets recomputed for each catalog with budgets held fixed from the smallest catalog to compare how restrictive the target budgets remain as catalogs grow. For Amazon, setting each query’s budget from its minimum feasible cost using a fixed query specific multiplier keeps the share of tight budget queries (cheapest feasible assignment above 80% of the target budget) at 0.41 across catalog sizes. Holding the 110K budgets fixed reduces this share to near 0 at 2.14M, where the median minimum feasible cost is $0.45\times$ its 110K value. Under these fixed budgets, mean recall is 0.22 for PBS and 0.38 for PTS.

For Airbnb, we generate 100 queries on the 29K catalog and use nested catalogs of 29K, 58K, 115K, and 231K listings with the same relative city sizes. Setting each query’s budget from its minimum feasible cost using a fixed query specific multiplier keeps the tight budget share at 0.71 across catalog sizes. Holding the 29K budgets fixed reduces this share to 0.01 at 231K. As catalogs grow, holding budgets fixed makes fewer queries tight. Recomputing budgets maintains the tight budget share at each size.

A9.3 Qualitative Amazon Example

This case study illustrates how the Pareto frontier represents trade-offs among shopping objectives. Table A2 shows six assignments from the frontier of an Amazon query with four requirements and a target budget of \$30. Its complete frontier contains 117 points. Among the six assignments shown, row 1 is the least expensive and stays within budget, with solution quality 0.821. The other rows illustrate how greater cost overhead can accompany higher solution quality or total rating. Rows 3 and 5 reach solution quality

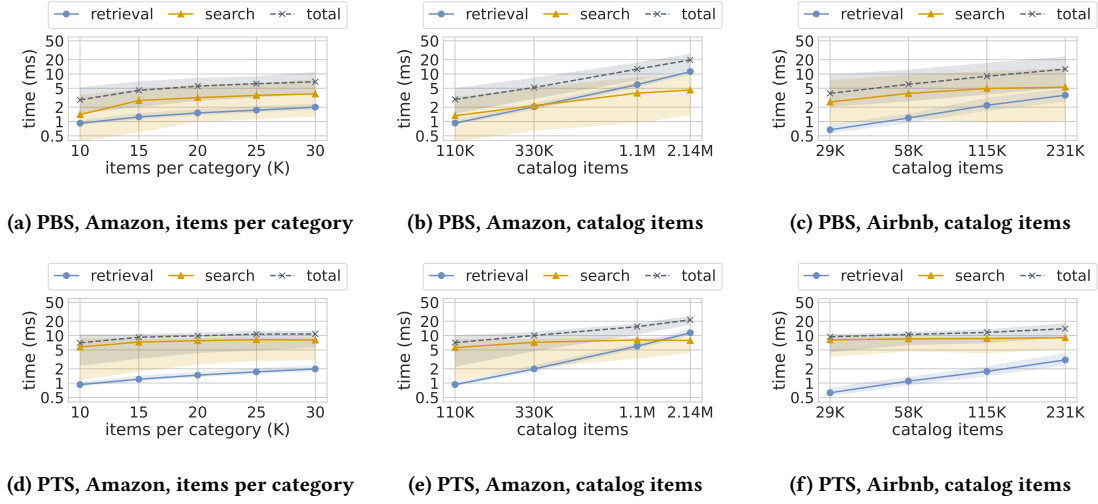


Figure A4: Median retrieval, search, and total latency across catalog sizes for PBS and PTS. Bands show interquartile ranges.

Table A1: Sampled host memory increase during search on Amazon. Values subtract memory after candidate loading from peak process memory. Sampling at 1 ms intervals may miss shorter peaks.

Catalog	PBS	PTS	ϵ -ILP ($q=4$)	ϵ -ILP ($q=16$)	PBS unbounded	DP
110K	0.49 MB	1.08 MB	5.54 MB	9.28 MB	0.79 MB	0.29 MB
330K	0.64 MB	1.96 MB	8.44 MB	14.08 MB	2.35 MB	0.63 MB
1.1M	0.80 MB	1.93 MB	17.24 MB	26.65 MB	7.39 MB	0.96 MB
2.14M	0.81 MB	1.99 MB	29.52 MB	43.91 MB	7.94 MB	1.00 MB

Table A2: Representative Pareto CGQ frontier on Amazon. ΔC denotes cost overhead.

#	req. 1	req. 2	req. 3	req. 4	Q	C	ΔC	R	found by
1	Seasky Buckta. (\$8)	Set of 6 Smal. (\$7)	Stove Top Cov. (\$7)	Slow Feeder A. (\$6)	0.821	\$27.77	\$0.00	4.03	PBS, PTS
2	Rapala Rattli. (\$6)	Sparkling Cry. (\$7)	Stove Top Cov. (\$7)	Anipaw Silico. (\$13)	0.831	\$33.46	\$3.46	3.95	PBS, PTS
3	S-Waver Joint. (\$10)	Sparkling Cry. (\$7)	Stove Top Cov. (\$7)	POPETPOP 2-in. (\$14)	0.842	\$38.73	\$8.73	3.48	PTS
4	Moose Bait Fr. (\$11)	Fully Lined F. (\$12)	Stove Top Cov. (\$7)	Kalinin Large. (\$9)	0.815	\$38.95	\$8.95	4.38	PBS, PTS
5	S-Waver Joint. (\$10)	Youme Banana. (\$8)	Stove Top Cov. (\$7)	MRECHIR Eleva. (\$16)	0.842	\$41.46	\$11.46	3.60	PTS
6	Couple Gifts. (\$10)	Fully Lined F. (\$12)	Stove Top Cov. (\$7)	Dr. Pet Ceram. (\$16)	0.820	\$44.86	\$14.86	4.42	PTS

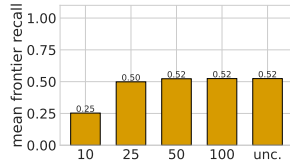


Figure A5: Effect of the candidate cap on PTS frontier recall at 2.14M Amazon items.

0.842 at cost overheads of \$8.73 and \$11.46, while row 6 reaches total rating 4.42 at an overhead of \$14.86. At the default setting, PBS recovers 75 of the 117 frontier points, and PTS recovers all 117. Rows 3, 5, and 6 are found only by PTS. All six bundles include the

same \$7 stove top cover, while the three PTS-only bundles pair it with fourth items costing \$14 to \$16. With a beam width of 100, PBS discards the partial assignments leading to these rows before the final requirement, whereas PTS explores them. PTS returns its first displayed point after 0.40 ms, compared with 3.6 ms for PBS. A single weighted objective returns one assignment for a given set of weights, so presenting these tradeoffs requires separate runs with different weights. The frontier presents these options directly to a user who is willing to exceed the target for a better bundle.

A9.4 Matching Threshold and Candidate Reduction

We vary the matching threshold to measure its effect on frontier recovery, as shown in Figure A6. We also compare PBS with and without candidate reduction using candidate evaluations, recall, and

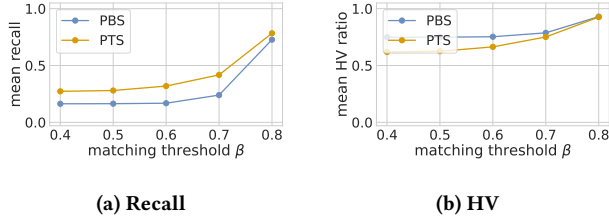


Figure A6: Mean frontier recall and HV ratio against the matching threshold β on Amazon.

HV. PBS and PTS return a feasible assignment on all 100 queries from $\beta = 0.4$ through $\beta = 0.8$.

Mean frontier recall at $\beta = 0.4$ is 0.163 for PBS and 0.273 for PTS. Both methods reach median recall 1 only at the default $\beta = 0.8$. Lowering β inflates the candidate sets, and the fixed default budgets then cover only a small share of the enlarged search space. Mean HV remains at least 0.74 for PBS and 0.61 for PTS over this range, indicating that the recovered points retain more of the dominated volume than of the reference frontier points. At $\beta = 0.9$, every query has a requirement with no eligible item and is therefore infeasible after retrieval. Recall and HV are undefined for all 100 queries at this setting. For PBS, removing candidate reduction leaves recall and hypervolume ratio identical on every query. The median paired ratio of candidate evaluations without reduction to those with reduction is 4.4 at $\beta = 0.8$ and 213 at $\beta = 0.6$. Candidate reduction lowers candidate evaluations without changing the frontier metrics on this workload.

A9.4.1 Latency and Memory on Airbnb. We compare PBS, PTS, and PBS without beam truncation on the 100 Airbnb queries as the matching threshold varies in Figure A7. Lowering β from 0.8 to 0.7 increases the median latency of PBS without beam truncation from 21 to 647 ms and its p90 latency from 167 to 3,196 ms. At $\beta = 0.7$, PBS and PTS have median latencies of 139 and 102 ms and p90 latencies of 256 and 160 ms. Bounded search reduces latency at this threshold, although both methods exceed the assumed 100 ms p90 target.

Over the same range, the sampled peak memory increase of PBS without beam truncation rises from 8.1 to 62.0 MB. PBS stays between 1.2 and 1.3 MB, and PTS between 1.8 and 2.8 MB.

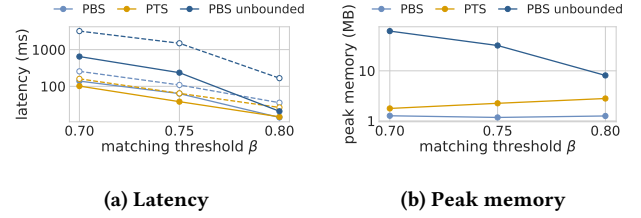


Figure A7: Latency and peak memory of PBS, PTS, and PBS without beam truncation as the matching threshold varies on Airbnb. Latency includes retrieval, with solid lines for medians and dashed lines for 90th percentiles. Memory is the sampled peak increase of each method's process over its loaded candidate sets across the 100 queries.